第一章
时域离散信号与系统
Discrete-time Signal and System

2025-11-4

信号的基本分类

- 根据信号的自变量"x"和函数值"y"对信号进行划分
  - 三种类型的信号
    - 模拟信号（时域连续信号）（x，y连续）
    - 时域离散信号（离散时间信号）（x离散，y连续）
    - 数字信号（ x，y离散）

信号的基本分类

◆三类信号的关系　　系统也一样分为三类

模拟信号（时域连续信号）$x_a(t)$ —按时间间隔T等间隔采样→ 时域离散信号 $x_a(nT)$ —函数值进行幅度量化→ 数字信号 $x(n)$

例：

$$x_a(t)=0.9\sin(50\pi t) \xrightarrow{T=0.005s} x_a(nT)=\{L\ ,0.0,0.6364,0.9,L\ \}$$

$$\xrightarrow{\text{函数值量化}} x(n)=\{L\ ,0.000,0.101,0.111,L\ \}$$

1.2 时域离散信号—序列

对模拟信号$x_a(t)$进行等间隔抽样，得到

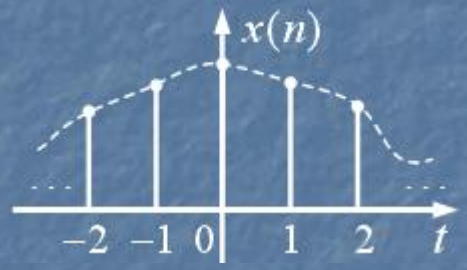
$$x_a(t)\big|_{t=nT} = x_a(nT) \quad -\infty < n < \infty$$

其中T为抽样间隔， n为整数。

- 序列$x(n)$定义为

$$x(n) = x_a(nT)$$

当n=…-2,-1,0,1,2…



$$x(n) = \{\text{L} \ ， \ x_a(-2T), x_a(-T), x_a(0), x_a(T), \text{L} \ \}$$

1.2 时域离散信号—序列

- 序列的表示方法：
  - 集合
  - 数学表达式（公式）
  - 波形（图形）

集合表示方法

$$x(n)=\{x_n;n=\mathrm{L}\ \ ,-2,-1,0,1,2,\mathrm{L}\ \}$$

例如：

$$x(n)=\{1,2,3,5,4,2;n=-2,-1,0,1,2,3\}$$

$$x(n)=\{1,2,\underline{3},5,4,2\}$$

缺点？ 优点？

知识点练习

- 将以下信号表示为集合形式的序列

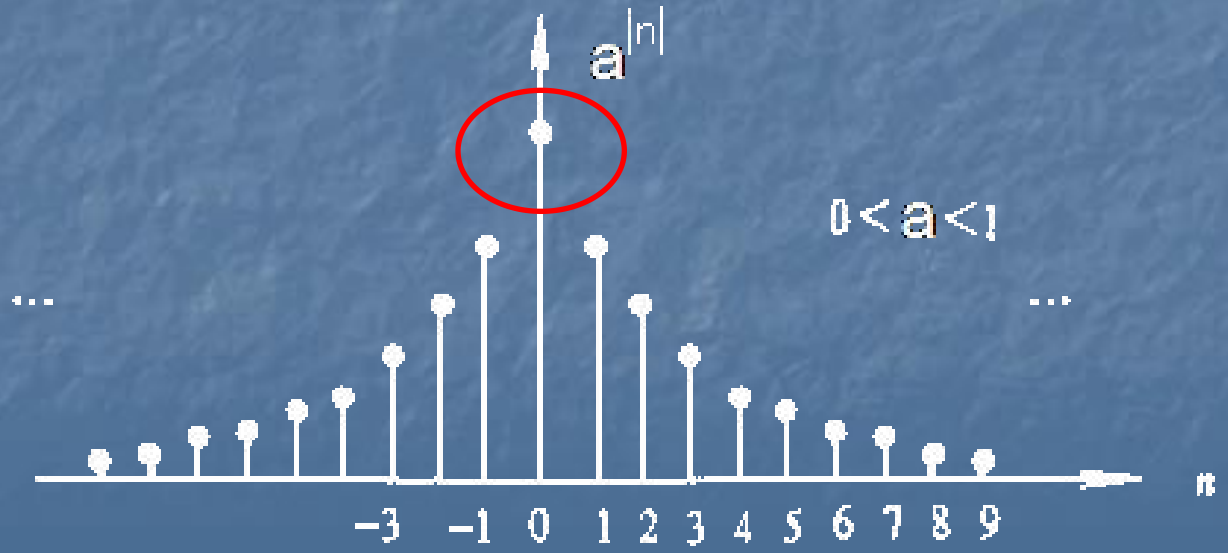
$$x(-2)=1, x(-1)=2, x(0)=3, x(1)=4, x(2)=5$$

$$x(n)=\{1,2,\underline{3},4,5\}$$

公式法与图形法

- （实）指数序列， 例如：

$$x(n)=a^{|n|} \qquad 0<a<1,-\infty<n<\infty$$



MATLAB表示序列

例：x(n)={-0.9511,-0.5878,0.0000, 0.5878, 0.9511}

表示： n=-2:2;

x=[-0.9511,-0.5878,0.0000, 0.5878, 0.9511]

由于x(n)是5个正弦序列的采样值

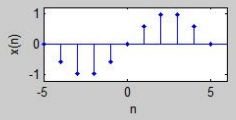
另表示：n=-2:2;

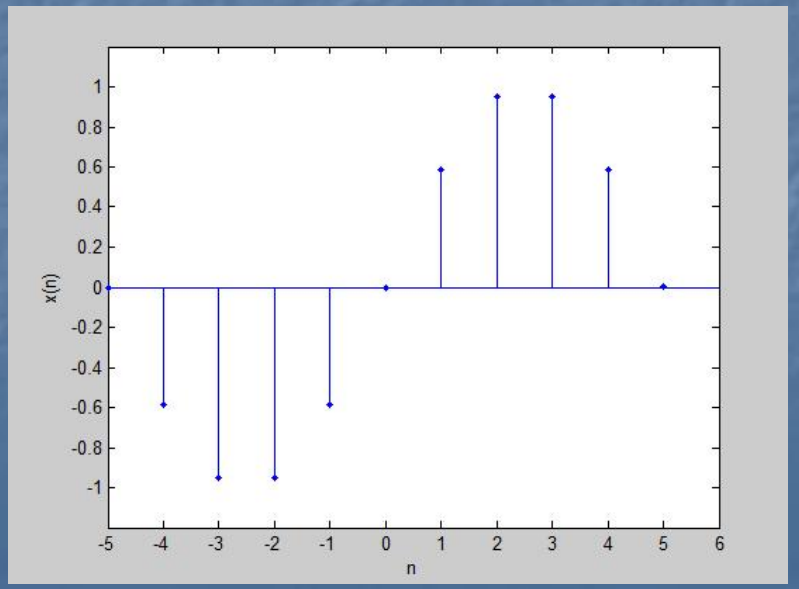
x=sin(pi*n/5)

完整的绘图程序？

MATLAB表示序列

```
n=-5:5;
x=sin(pi*n/5);
subplot(3,2,1);%图的分布是3行，2列，从左到右，从上到下
stem(n,x,'.');%在n的位置画出x的值，格式为‘.’
line([-5,6],[0,0])%画出（-5,0）到（6,0）的一条直线
axis([-5,6,-1.2,1.2]);%坐标轴范围
xlabel('n');ylabel('x(n)');%x，y轴标签
```





Python表示序列

simple_sin_signal_plot.py

```python
import numpy as np
import matplotlib.pyplot as plt

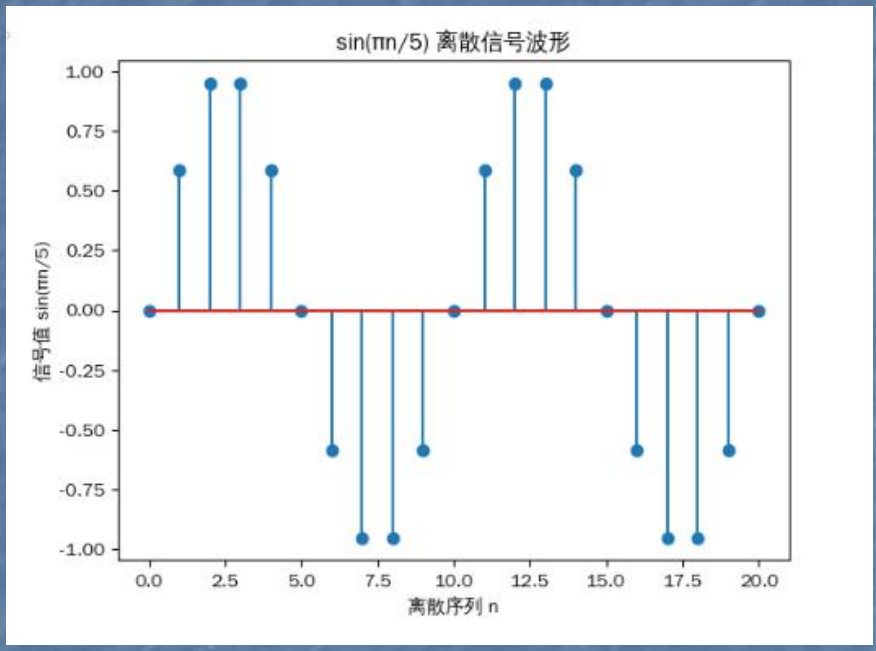
# 1. 生成离散序列n（0到20，包含2个周期，周期为10）
n = np.arange(0, 21)  # 离散点：0,1,2,...,20

# 2. 计算sin(πn/5)信号值
signal = np.sin(np.pi * n / 5)

# 3. 绘制离散信号（用stem函数体现离散特性）
plt.stem(n, signal)

# 4. 基础标注（明确坐标轴和标题）
plt.title('sin(πn/5) 离散信号波形')
plt.xlabel('离散序列 n')
plt.ylabel('信号值 sin(πn/5)')

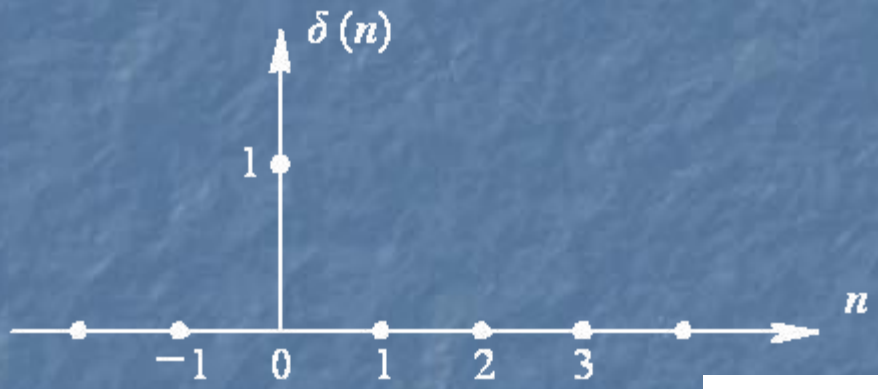
# 5. 显示图形
plt.show()
```



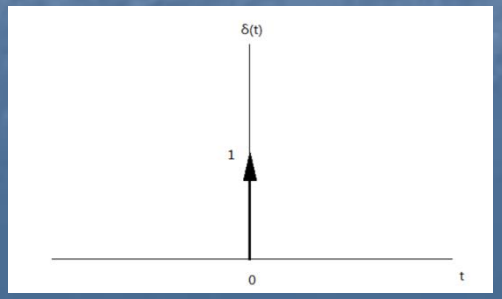
1.2.1 常用的典型序列

1）单位脉冲序列

$$\delta(n)=\begin{cases}1 & n=0\\ 0 & n\neq 0\end{cases}$$



对比：单位冲激函数 $\delta(t)$



硬件类比：单脉冲触发器

按下按键时（n=0）产生一个 1V 的瞬间电压，松开后（n≠0）电压回归 0V。

工程场景：数字通信中的“同步信号”：

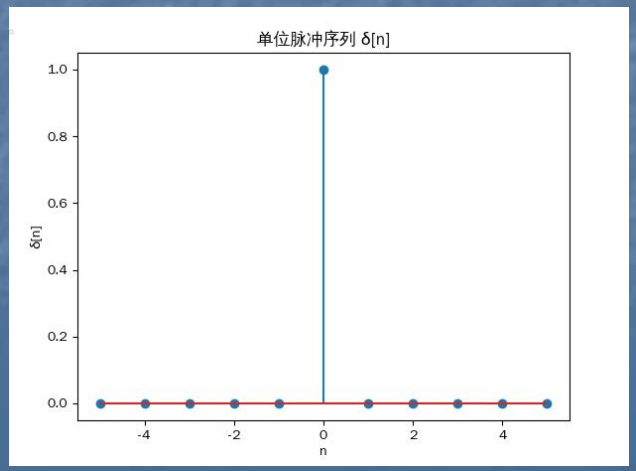
用 δ[n] 作为帧同步标记（每帧开头发一个 δ[n]，接收端识别后对齐数据）

代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

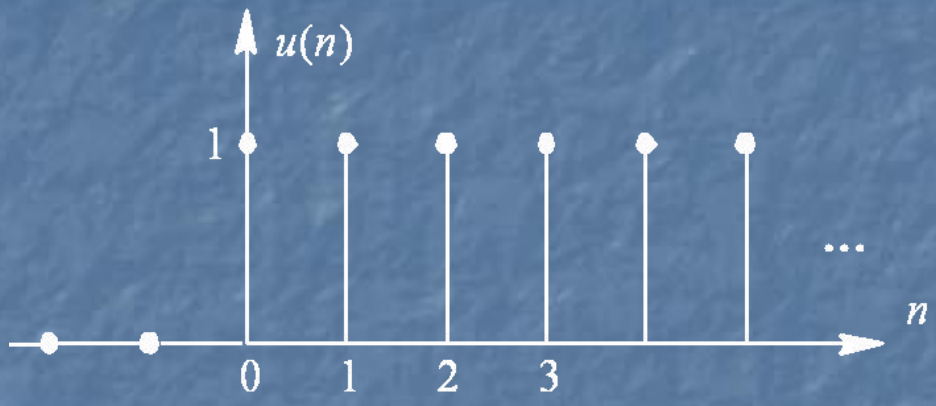
n = np.arange(-5, 6)  # 取n从-5到5，体现"仅n=0非零"
delta = np.where(n == 0, 1, 0)  # 单位脉冲序列

plt.stem(n, delta)
plt.title('单位脉冲序列 δ[n]')
plt.xlabel('n')
plt.ylabel('δ[n]')
plt.show()
```



2）单位阶跃序列

$$u(n)=\begin{cases}1 & n\geq 0\\ 0 & n<0\end{cases}$$



掌握：与单位脉冲序列的关系

$$\delta(n)=u(n)-u(n-1)$$

$$u(n)=\sum_{m=0}^{\infty}\delta(n-m)=\delta(n)+\delta(n-1)+\delta(n-2)+\ldots$$

$$=\sum_{k=-\infty}^{n}\delta(k)$$

硬件类比：单刀单掷开关

开关闭合前（n<0）电路无电流（输出 0），闭合后（n≥0）电流持续导通。

工程场景：信号“持续激励”控制：

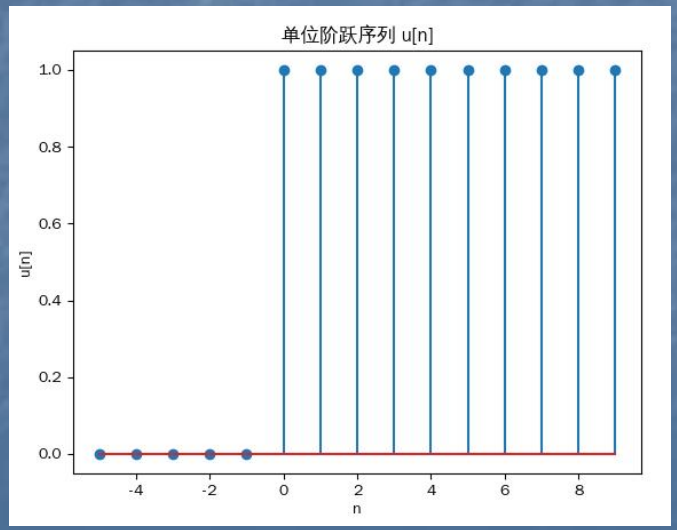
给扬声器输入 u [n]（持续 1 的信号），对应扬声器“通电后持续发声”

代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

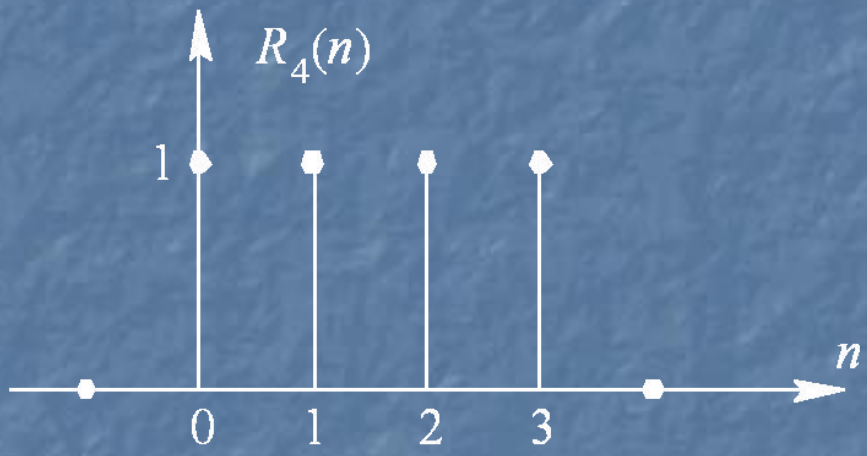
n = np.arange(-5, 10)  # n从-5到10，覆盖"开关前后"
u = np.where(n >= 0, 1, 0)  # 单位阶跃序列

plt.stem(n, u)
plt.title('单位阶跃序列 u[n]')
plt.xlabel('n')
plt.ylabel('u[n]')
plt.show()
```



3）矩形序列$R_N(n)$

$$R_N(n)=\begin{cases}1 & 0\le n\le N-1\\ 0 & \text{其它n}\end{cases}$$



与其它序列 $\delta(n),u(n)$ 的关系

$$R_N(n)=u(n)-u(n-N)$$

$$R_N(n)=\sum_{m=0}^{N-1}\delta(n-m)=\delta(n)+\ldots+\delta[n-(N-1)]$$

硬件类比：相机的快门信号

按下快门后，快门仅在 N 个离散时间点内打开。

工程场景：雷达的“发射脉冲”：

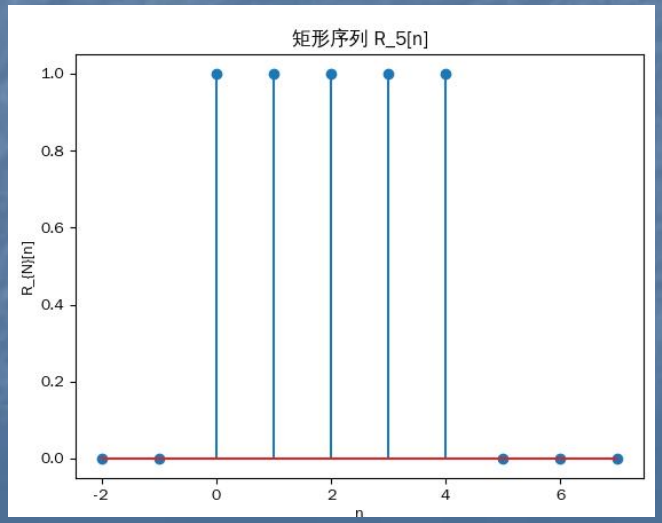
雷达发射的信号是矩形序列（仅在 N 个时间点内发射电磁波，避免持续耗能）

代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

N = 5  # 矩形序列长度
n = np.arange(-2, 8)  # 覆盖序列前后
rect = np.where((n >= 0) & (n < N), 1, 0)  # 矩形序列

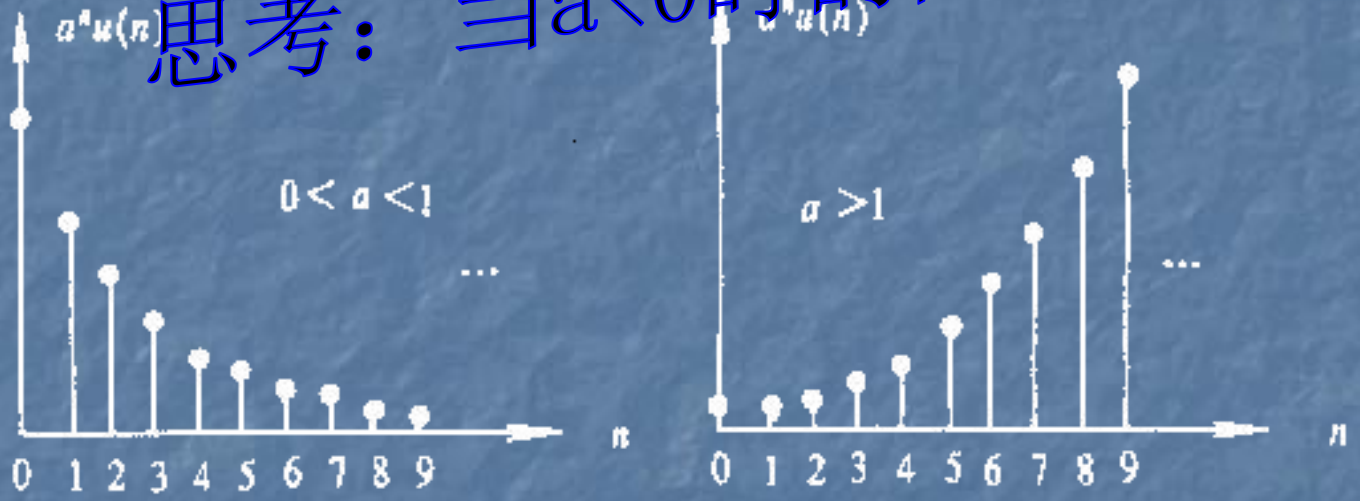
plt.stem(n, rect)
plt.title(f'矩形序列 R_{N}[n]')
plt.xlabel('n')
plt.ylabel('R_{N}[n]')
plt.show()
```



4）实指数序列 $x(n)=a^n u(n)$　　a 为实数

$|a|<1$ 收敛　　$|a|>1$ 发散

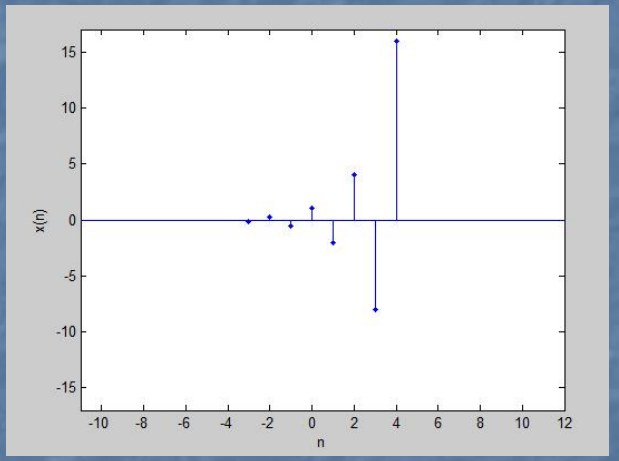
思考：当a<0时的图形？



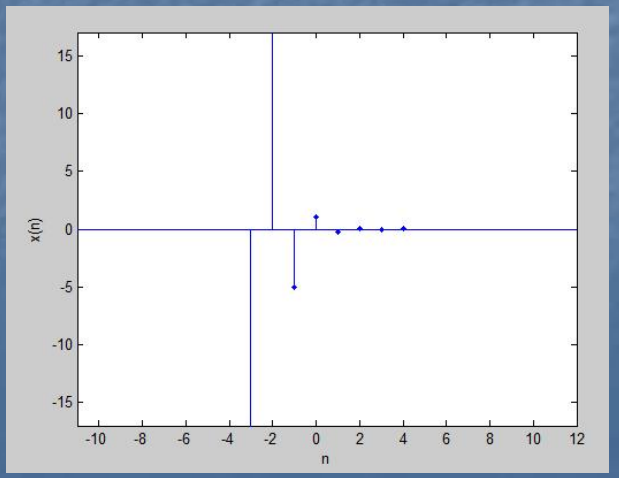
4）实指数序列 $x(n)=a^n u(n)$　　a 为实数

$a<0$

```
n=-3:4;a=-2;
x=a.^n;
subplot(1,1,1);
stem(n,x,'.');
line([-11,12],[0,0])
axis([-11,12,-17,17]);
xlabel('n');ylabel('x(n)');
```




```
n=-3:4;a=-0.2;
x=a.^n;
subplot(1,1,1);
stem(n,x,'.');
line([-11,12],[0,0])
axis([-11,12,-17,17]);
xlabel('n');ylabel('x(n)');
```



硬件类比：“随时间渐变”的电子元件

对应“RC 电路的充、放电过程”—— 给电容充电到 1V（初始值 1），断开电源后，电容通过电阻放电，电压随时间逐渐下降。

工程场景：信号衰减

无线通信中“信号随距离的衰减”—— 手机离基站越远（n增大，可理解为“距离步长”），接收信号强度按指数规律下降（a<1），比如 a=0.9，每远离 100 米，信号强度变为之前的 90%；

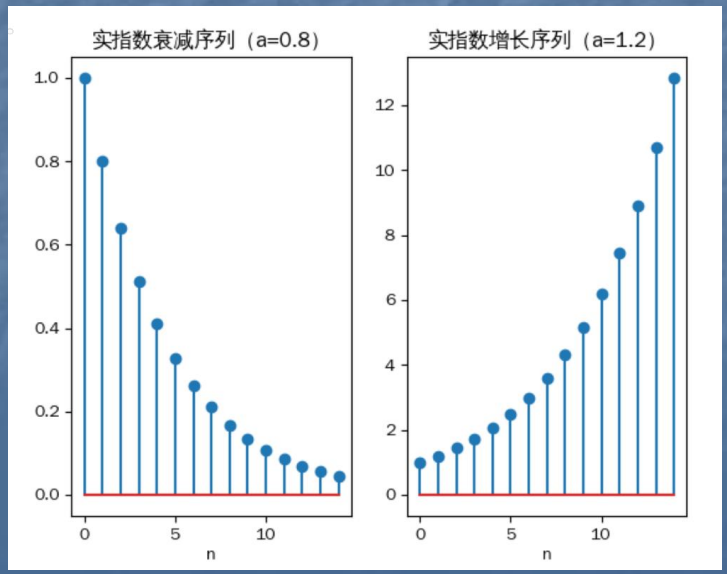
代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(0, 15)  # 取15个离散点，看趋势
a_decay = 0.8  # 衰减系数（0<a<1）
a_grow = 1.2   # 增长系数（a>1）
u = np.where(n >= 0, 1, 0)  # 阶跃序列，保证n<0时为0

# 生成两种实指数序列
x_decay = (a_decay ** n) * u  # 衰减序列
x_grow = (a_grow ** n) * u    # 增长序列

# 画两个图对比
plt.subplot(1,2,1)
plt.stem(n, x_decay)
plt.title(f'实指数衰减序列（a={a_decay}）')
plt.xlabel('n')
plt.subplot(1,2,2)
plt.stem(n, x_grow)
plt.title(f'实指数增长序列（a={a_grow}）')
plt.xlabel('n')
plt.show()
```



5）正弦序列（余弦序列）

$x(n) = A\sin(\omega n + \phi)$　　ω：数字域频率

模拟正弦信号：

$x_a(t) = A\sin(\Omega t + \phi)$　　Ω：模拟域频率

$x(n) = x_a(t)\big|_{t=nT} = A\sin(\Omega nT + \phi)$　　T：采样周期

$\omega = \Omega T$

数字域频率表示序列变化的速率，也表示相邻两个采样点之间的相位变化弧度数。（ ω 的物理含义）

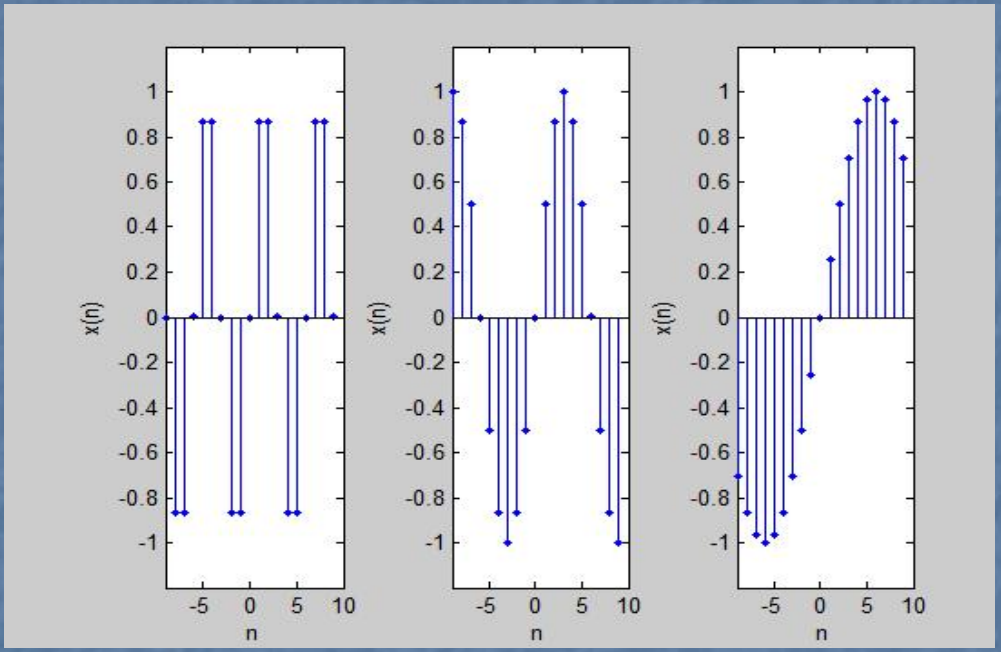
$\omega = \Omega T = \Omega / F_s$　　F_s：采样频率

数字域频率是模拟域频率对采样频率的归一化频率。

5）正弦序列（余弦序列）

数字域频率表示序列变化的速率，也表示相邻两个采样点之间的相位变化弧度数。（ ω 的物理含义）

```
n=-9:9;
x=sin(pi*n/3);
x1=sin(pi*n/6);
x2=sin(pi*n/12);
subplot(1,3,1);%图的分布是3行，2列，从左到右，从上到下
stem(n,x,'.');%在n的位置画出x的值，格式为‘.’
axis([-9,10,-1.2,1.2]);%坐标轴范围
xlabel('n');ylabel('x(n)');%x，y轴标签
subplot(1,3,2)
stem(n,x1,'.');
xlabel('n');ylabel('x(n)');%x，y轴标签
axis([-9,10,-1.2,1.2]);%坐标轴范围
subplot(1,3,3)
stem(n,x2,'.');
axis([-9,10,-1.2,1.2]);%坐标轴范围
xlabel('n');ylabel('x(n)');%x，y轴标签
```



ω1 =pi/3　　ω2 =pi/6　　ω3 =pi/12

硬件类比：电压 / 电流波动

对应“交流电源的电压波形”—— 家用 220V 交流电是正弦波（时域连续），若对其抽样（比如每 0.001 秒采一个点），得到的就是“离散正弦序列”。

工程场景：通信信号载波

无线通信（如手机、广播）用“高频正弦序列”作为“载波”—— 把语音 / 数据信号“叠加”在正弦载波上，通过天线发射（比如 FM 广播的载波频率是 88-108MHz，本质是高频正弦波）

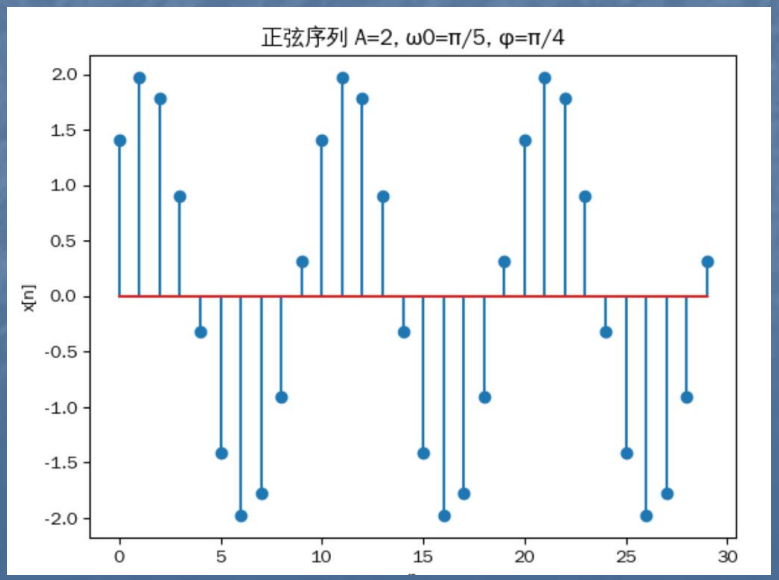
代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(0, 30)  # 足够长，体现周期性
A = 2           # 幅度（波动的最大范围）
omega0 = np.pi/5  # 角频率（决定周期，周期=2π/omega0=10）
phi = np.pi/4     # 初相位（决定起始位置）

# 生成正弦序列
x_sin = A * np.sin(omega0 * n + phi)

plt.stem(n, x_sin)
plt.title(f'正弦序列 A={A}, ω0=π/5, φ=π/4')
plt.xlabel('n')
plt.ylabel('x[n]')
plt.show()
```



6）复指数序列（复正弦）

$$x(n)=e^{(\sigma+j\omega_0)n}=e^{\sigma n}\cdot e^{j\omega_0 n}$$

$$=e^{\sigma n}[\cos(\omega_0 n)+j\sin(\omega_0 n)]$$

ω_0为数字域频率

例：$x(n)=0.9^n e^{j\frac{\pi}{3}n}$

复指数序列以2π为周期

$$e^{j(\omega_0 n+2\pi M)n}=e^{j(\omega_0 n)n}$$

$$\cos(\omega_0 n+2\pi M)n=\cos(\omega_0 n)$$

$$\sin(\omega_0 n+2\pi M)n=\sin(\omega_0 n)$$

7) 周期序列

若对所有n存在一个最小的正整数N，满足

$$x(n)=x(n+N) \quad -\infty<n<\infty$$

则称序列$x(n)$为周期性序列，周期为N。

- 对正弦序列的周期性：

$$x(n)=A\sin\left(\omega_0 n+\varphi\right)$$

$$x(n+N)=A\sin\left(\omega_0 n+\omega_0 N+\varphi\right)$$

当$N=\left(2\pi/\omega_0\right)k$，正弦序列为周期的。

硬件类比：LED 流水灯的“循环亮灭”

4 个 LED 按“亮、灭、灭、灭→灭、亮、灭、灭→...→灭、灭、灭、亮”循环（周期 N=4），每个 LED 的“亮 = 1，灭 = 0”信号就是周期序列。

工程场景：数据传输同步

串口通信（如 USB）中的“同步时钟序列”—— 发送端和接收端用同一个周期序列（如 N=8），每 8 个点同步一次数据，避免传输错位。

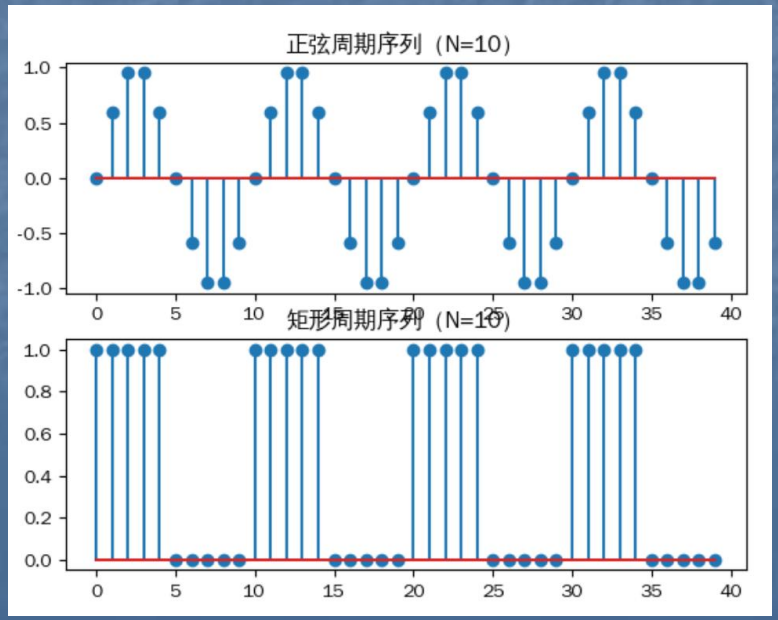
代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

n = np.arange(0, 40)  # 取4个周期（N=10）
N = 10  # 最小正周期

# 1. 正弦周期序列（之前的正弦序列，周期N=10）
x_sin_period = np.sin(np.pi/5 * n)
# 2. 矩形周期序列（0-4为1，5-9为0，循环）
x_rect_period = np.where(n % N < 5, 1, 0)  # n%N求余数，实现循环

# 对比两种周期序列
plt.subplot(2,1,1)
plt.stem(n, x_sin_period)
plt.title(f'正弦周期序列（N={N}）')
plt.subplot(2,1,2)
plt.stem(n, x_rect_period)
plt.title(f'矩形周期序列（N={N}）')
plt.show()
```



判断周期性，三种情况：

（1）$2\pi/\omega_0$ 为整数，则周期为 $2\pi/\omega_0$

（2）$2\pi/\omega_0$ 不为整数，但为有理数，
则周期为 $2\pi/\omega_0=P/Q$,取k=Q,N=P

（3）$2\pi/\omega_0$ 为无理数，则不是周期的

- 对于复指数序列 $e^{j\omega_0 n}$ 有同样的分析方法

例. 判断如下序列的周期性

(1) $\sin(\frac{\pi}{4}n)$　(2) $\sin(\frac{4\pi}{5}n)$　(3) $e^{j(\frac{n}{6}-\pi)}$

解：(1) $2\pi/\omega_0 = 2\pi/(\frac{\pi}{4}) = 8$　周期为8序列

(2) $2\pi/\omega_0 = 2\pi/(\frac{4\pi}{5}) = \frac{5}{2}$　周期为5序列

(3) $2\pi/\omega_0 = 2\pi/(\frac{1}{6}) = 12\pi$　非周期序列

8) 任意序列

对任意序列x(n)，可以用单位脉冲序列的移位加权和表示：

$$x(n)=\sum_{m=-\infty}^{\infty}x(m)\delta(n-m)$$

$$x(n)=-2\delta(n+2)+0.2\delta(n+1)+2\delta(n)+\delta(n-1)$$

exercise

1. 序列 $x(n)=\sin\left(\frac{48}{180}\pi n+\frac{\pi}{4}\right)$ 的周期是__15__。

2. 序列 $x(n)=e^{j\frac{3}{4}\pi n}$ 的周期是__8__。

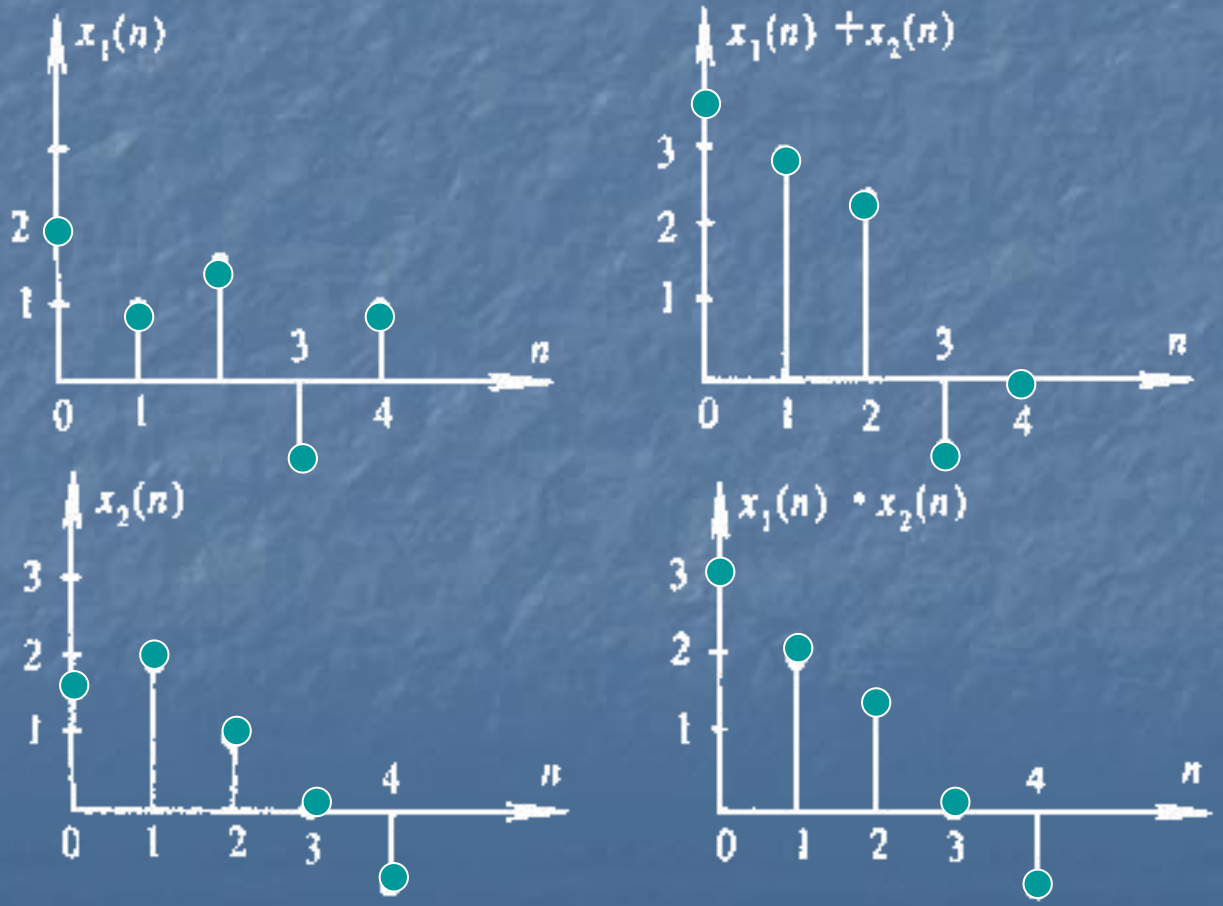
1.2.2 序列的运算

1）加法、乘法

$$x(n)=x_1(n)+x_2(n)$$

$$x(n)=x_1(n)\cdot x_2(n)$$

两个或多个信号对应时刻的函数值相加或相乘。



工程场景：多频信号合成（加法）

音乐中的“和弦”就是多个单频正弦信号的叠加：比如 C 和弦包含“C 音（261.6Hz）、E 音（329.6Hz）、G 音（392Hz）”—— 每个音对应一个正弦序列，三者相加就是 C 和弦的音频信号序列。

代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

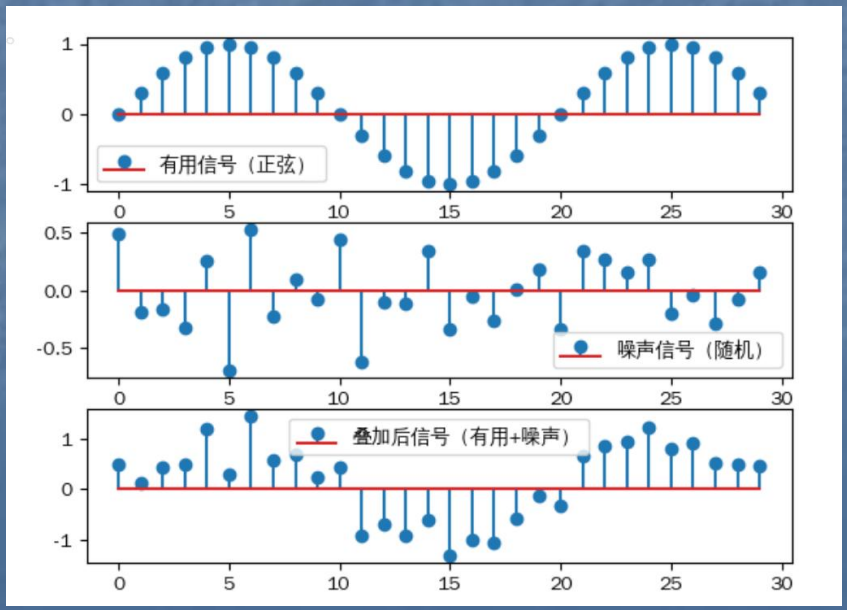
n = np.arange(0, 30)  # 离散序列n

# 1. 生成有用信号（比如低频正弦信号，模拟人声）
x1 = np.sin(np.pi/10 * n)  # 频率低，波形平缓

# 2. 生成噪声信号（随机序列，模拟环境噪声）
np.random.seed(1)  # 固定随机种子，结果可重复
x2 = 0.3 * np.random.randn(len(n))  # 噪声幅度0.3（比信号小，符合实际）

# 3. 序列加法：有用信号 + 噪声
y = x1 + x2

# 画图对比
plt.subplot(3,1,1)
plt.stem(n, x1, label='有用信号（正弦）')
plt.legend()
plt.subplot(3,1,2)
plt.stem(n, x2, label='噪声信号（随机）')
plt.legend()
plt.subplot(3,1,3)
plt.stem(n, y, label='叠加后信号（有用+噪声）')
plt.legend()
plt.show()
```



工程场景：信号调制（通信中“传远信号”的关键）（乘法）

低频信号（如语音，200Hz-3kHz）无法直接通过天线远距离传输，需要“乘上高频正弦序列（载波）”—— 这个过程叫“调制”，调制后的信号频率变高，能通过天线发射。比如广播电台的“FM 调制”，本质就是“语音序列 × 高频载波序列”。

代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

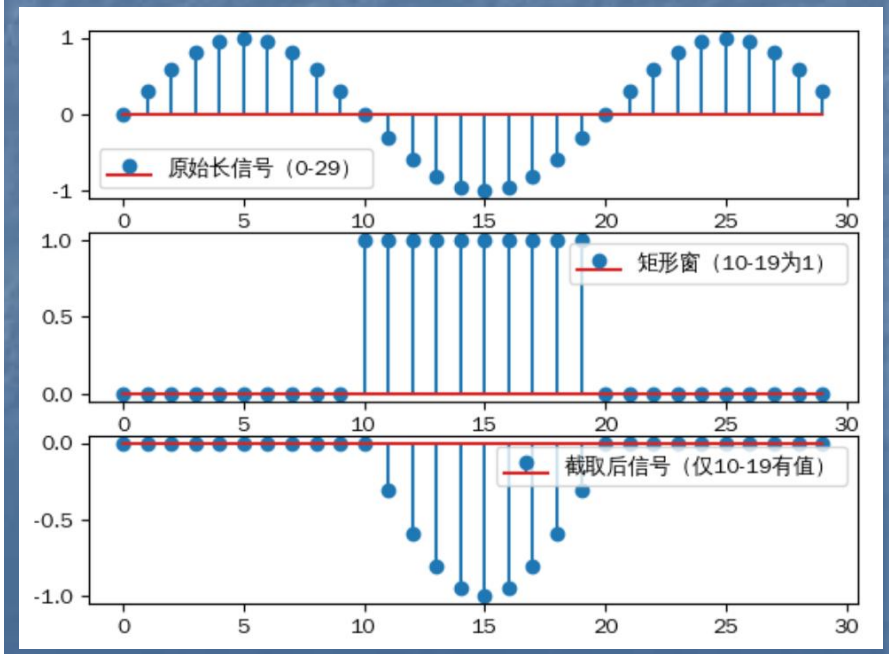
n = np.arange(0, 30)  # 长序列n（0-29）

# 1. 生成原始长信号（正弦序列，模拟长数据）
x1 = np.sin(np.pi/10 * n)

# 2. 生成矩形窗序列（中间10个点为1，其余为0，模拟“选段窗”）
window = np.where((n >= 10) & (n <= 19), 1, 0)  # 只保留n=10-19的信号

# 3. 序列乘法：原始信号 × 窗函数（截取中间片段）
y = x1 * window

# 画图对比
plt.subplot(3,1,1)
plt.stem(n, x1, label='原始长信号（0-29）')
plt.legend()
plt.subplot(3,1,2)
plt.stem(n, window, label='矩形窗（10-19为1）')
plt.legend()
plt.subplot(3,1,3)
plt.stem(n, y, label='截取后信号（仅10-19有值）')
plt.legend()
plt.show()
```



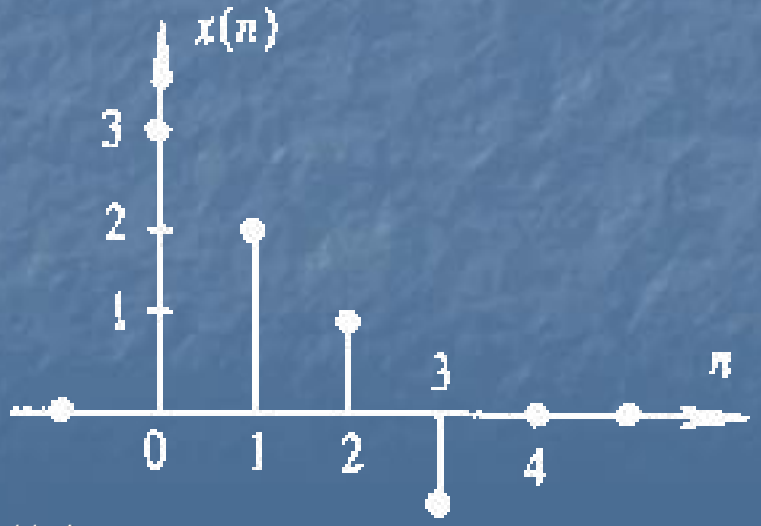
1.2.2 序列的运算

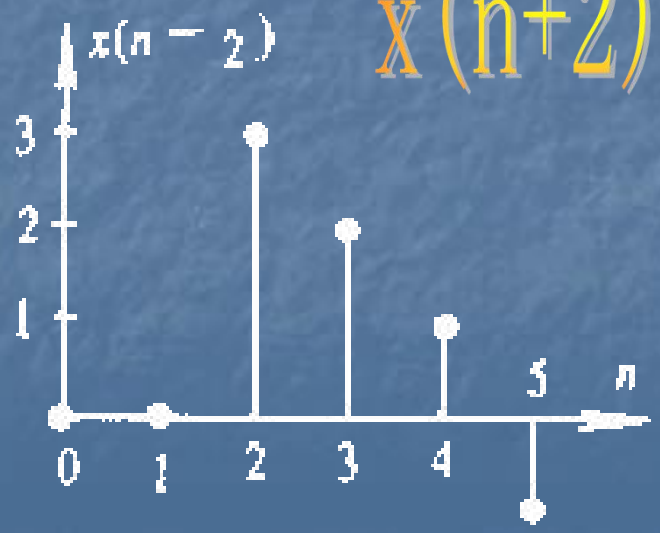
2）移位

已知序列x(n)，对于序列$x(n-n_0)$，

当$n_0>0$时，延时/右移n_0 x(n-m),右移（正数）m

当$n_0<0$时，超前/左移n_0 x(n+m),左移（负数）m





x(n+2)???

工程场景：延迟补偿（传感器）（移位）

两个温度传感器（A 和 B），A 离热源近，B 离热源远，B 的温度信号比 A “晚 3 个采样周期”出现。要对比两者的温度变化，需要把 A 的信号 “右移 3 位”（模拟延迟），或把 B 的信号 “左移 3 位”，让两者时序对齐。

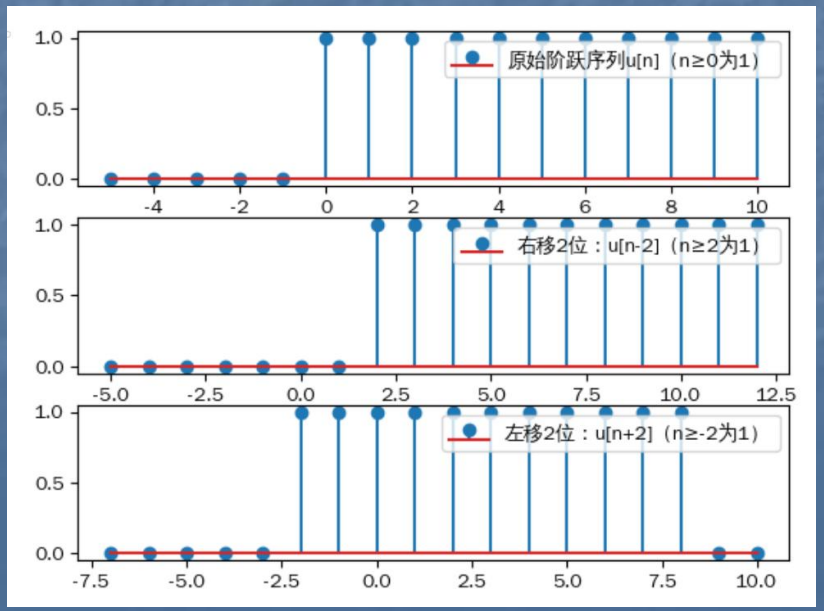
代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

# 原始序列：n从-5到10，阶跃序列u[n]（n≥0时为1）
n_original = np.arange(-5, 11)
x = np.where(n_original >= 0, 1, 0)

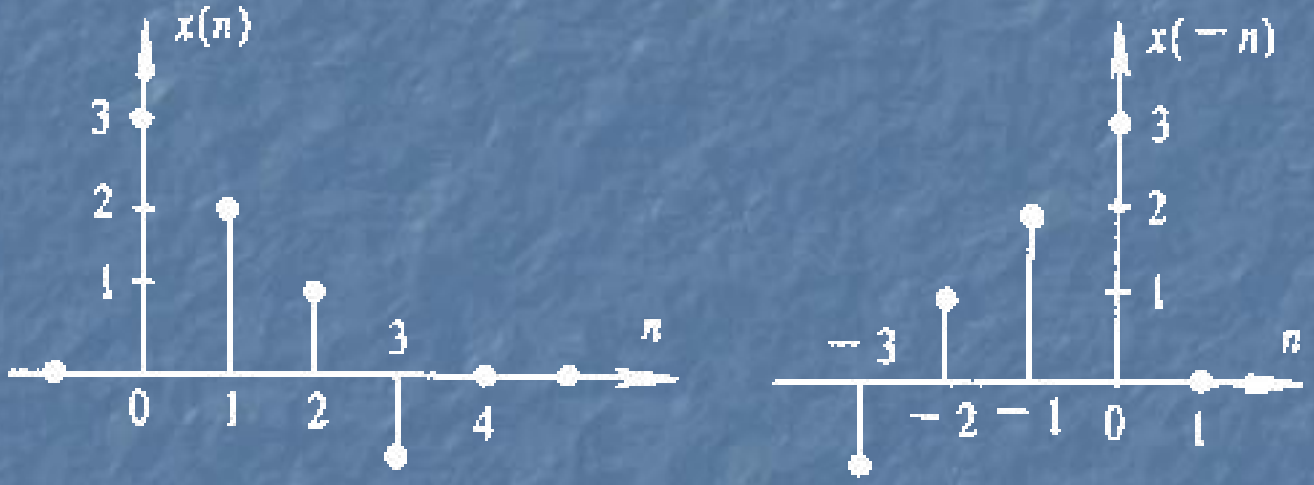
# 1. 右移2位：y1[n] = x[n-2]（信号往后移2个单位）
# 新n范围要包含移位后的信号（-5到12，覆盖右移后的终点）
n_shift_right = np.arange(-5, 13)
# 对每个新n，找对应的n-2是否在原始n中，在则取x值，否则0
y1 = np.zeros_like(n_shift_right)
for i, n in enumerate(n_shift_right):
    idx = np.where(n_original == (n - 2))[0]  # 找n-2在原始n中的位置
    if len(idx) > 0:
        y1[i] = x[idx[0]]

# 2. 左移2位：y2[n] = x[n+2]（信号往前移2个单位）
n_shift_left = np.arange(-7, 11)  # 新n范围包含左移后的起点（-7到10）
y2 = np.zeros_like(n_shift_left)
for i, n in enumerate(n_shift_left):
    idx = np.where(n_original == (n + 2))[0]  # 找n+2在原始n中的位置
    if len(idx) > 0:
        y2[i] = x[idx[0]]
```



3）翻转（反折）

序列x(-n)和x(n) 以n=0为对称轴镜像反转。



工程场景：雷达目标定位（回波信号匹配）（反折）

雷达发射一个“探测序列 x [n]”，遇到目标后反射回“回波序列 y [n]= x [n-l]”，因为是通过计算相关性来匹配发射和接收信号，从而找出延时，再来确定目标的距离，“发射序列反转”，再和“回波序列”对比（如计算相关性），就能确定目标的距离（反转后的匹配程度越高，距离越准）。

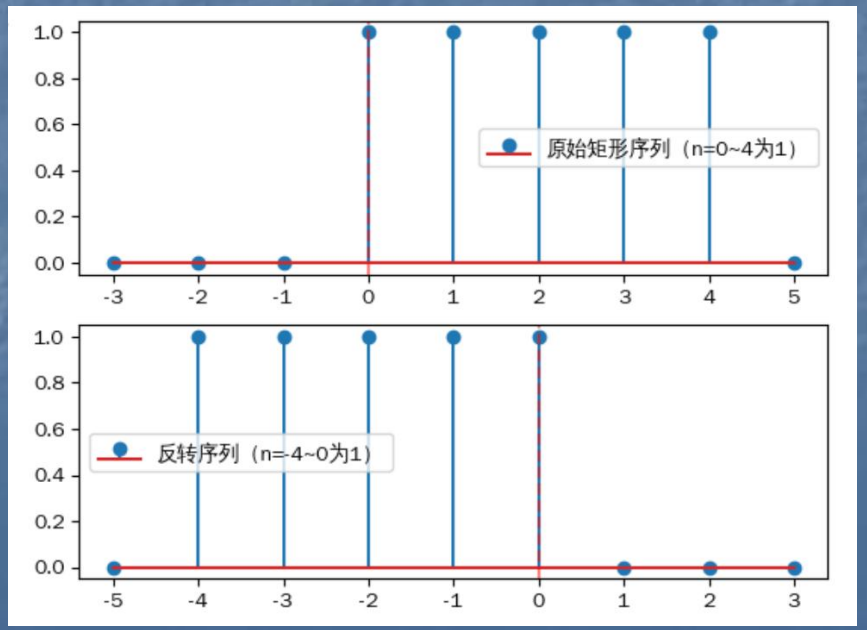
代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

# 原始序列：n从-3到5，矩形序列R5[n]（n=0~4时为1，其余为0）
n_original = np.arange(-3, 6)
x = np.where((n_original >= 0) & (n_original <= 4), 1, 0)

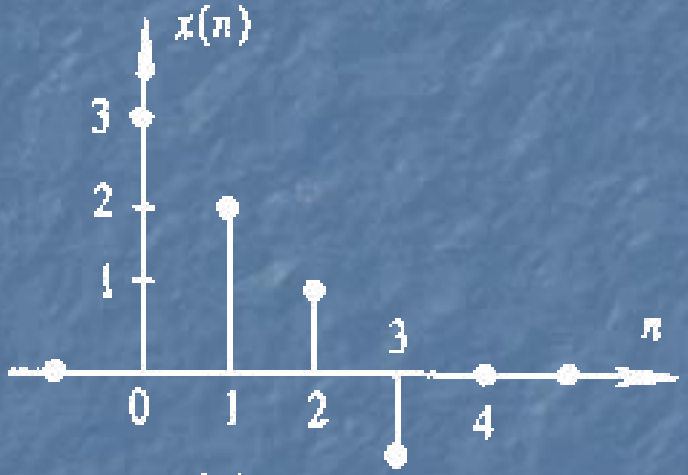
# 序列反转：y[n] = x[-n]
# 第一步：确定反转后的n范围（原始n是-3~5，反转后是-5~3）
n_reverse = np.arange(-5, 4)
# 第二步：对每个反转后的n，取x[-n]的值（-n在原始n中则取x值，否则0）
y = np.zeros_like(n_reverse)
for i, n in enumerate(n_reverse):
    idx = np.where(n_original == (-n))[0]  # 找-n在原始n中的位置
    if len(idx) > 0:
        y[i] = x[idx[0]]

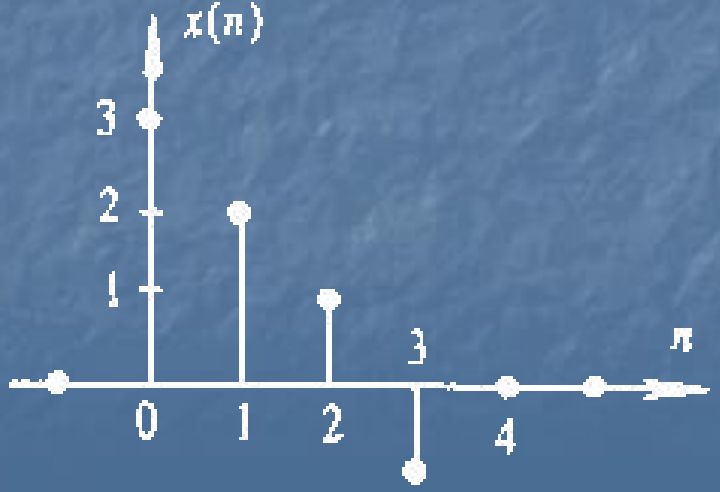
# 画图对比（加n=0的竖线，更易看到翻转轴）
plt.subplot(2,1,1)
plt.stem(n_original, x, label='原始矩形序列（n=0~4为1）')
plt.axvline(x=0, color='red', linestyle='--', alpha=0.5)  # 翻转轴
plt.legend()
plt.subplot(2,1,2)
plt.stem(n_reverse, y, label='反转序列（n=-4~0为1）')
plt.axvline(x=0, color='red', linestyle='--', alpha=0.5)  # 翻转轴
plt.legend()
plt.show()
```

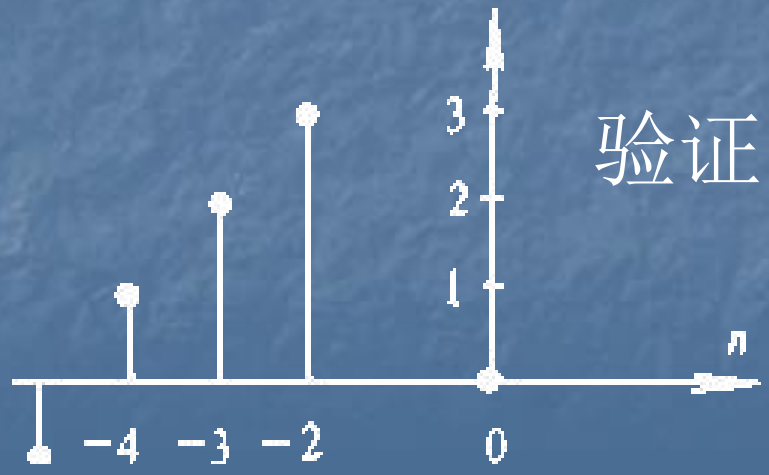


思考题

已知x(n)如下，画出x(-n-2)







验证？

4）尺度变换

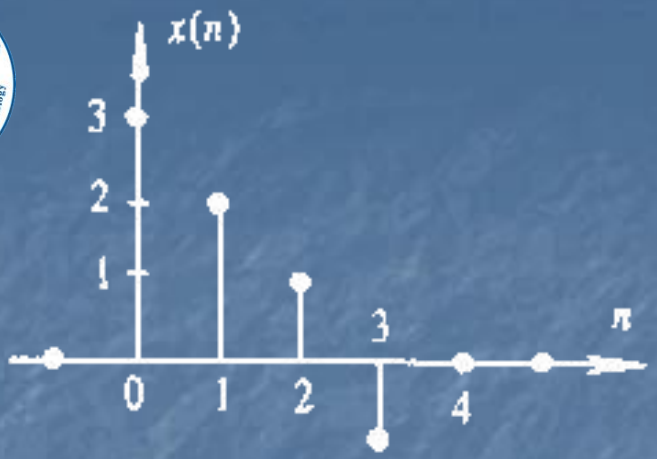
对于序列 $x(Mn)$，当 $M>1$时，抽取；

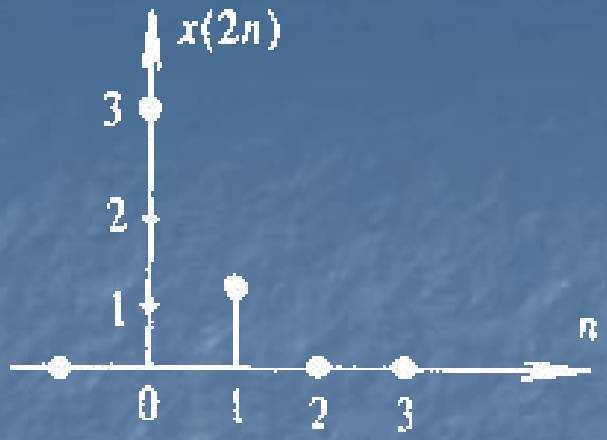
保留 n 为 M 的倍数的点

当 $0<M<1$ 时，插值。

在原始序列的每个点之间“插入 (1/M)-1 个 0”

离散序列的尺度变换，**本质**是“调整采样点的数量（密度）”，而非连续信号的“波形平滑拉伸”—— 这是工程中“改变信号采样率”的基础操作。

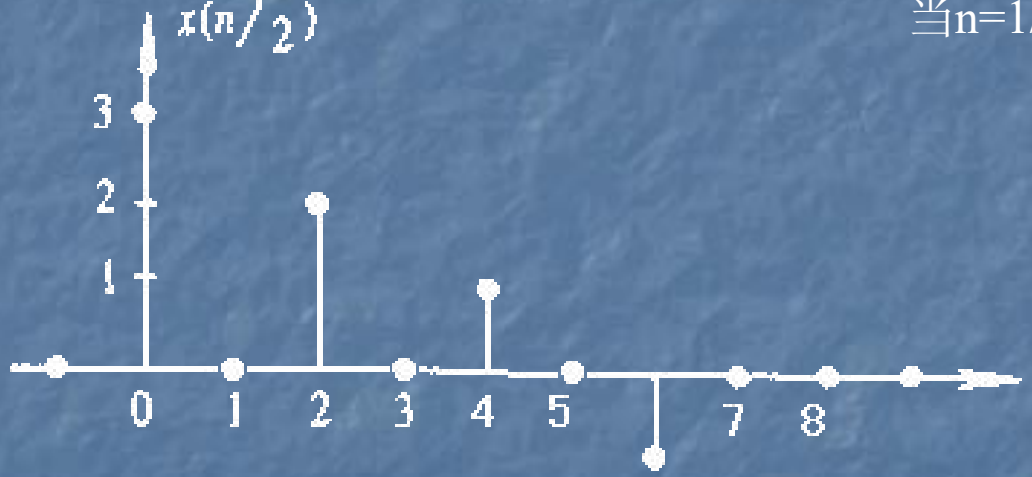




当n=0时,x(2n)=x(0)=3;

当n=1时,x(2n)=x(2)=1;

当n=1/2时,x(2n)=x(1)=2;但n为整数



当n=1时,x(n/2)=x(1/2)=?;

当n=2时,x(n/2)=x(1)=2;

当 $M<0$ 时？

工程场景：图像降采样（抽取）

采集到的图像信号采样率过高（1920×1080 像素），但手机屏幕仅需 640×360 像素（低分辨率）—— 此时需要对 1920×1080 信号做“k=3 的压缩”（每隔 2 个点取 1 个点，即取 n=0,3,6,... 的点），减少数据量，避免存储 / 传输压力（这就是工程中的“抽取降采样”）。

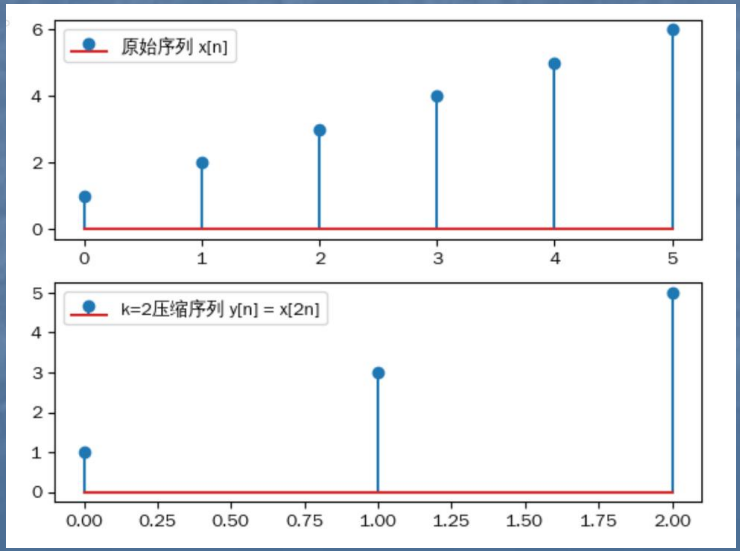
代码演示：

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. 原始序列：n从0到5，x[n] = [1,2,3,4,5,6]
n_original = np.arange(0, 6)
x = np.array([1, 2, 3, 4, 5, 6])

# 2. 尺度变换：k=2压缩（y[n] = x[kn]，取k的倍数点）
k = 2
# 压缩后的索引：取原始序列中n为k倍数的索引（0,2,4...）
idx_compress = np.arange(0, len(n_original), k)  # 步长为k
n_compress = np.arange(len(idx_compress))  # 压缩后的n（从0开始）
y = x[idx_compress]  # 抽取对应点

# 画图对比：压缩后仅保留原始序列的部分点
plt.subplot(2,1,1)
plt.stem(n_original, x, label='原始序列 x[n]')
plt.legend()
plt.subplot(2,1,2)
plt.stem(n_compress, y, label=f'k=2压缩序列 y[n] = x[2n]')
plt.legend()
plt.show()
```



工程场景：图像的“放大显示”（插值）

图像的每个像素是离散序列，放大时需要对像素序列做展宽（补零），再插值得到更多像素，避免放大后出现锯齿。

代码演示：

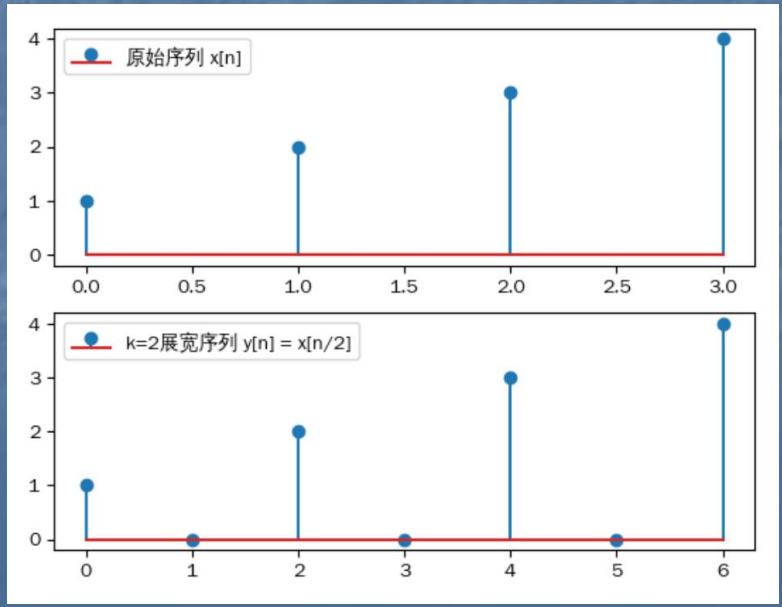
```python
import numpy as np
import matplotlib.pyplot as plt

# 1. 原始序列：n从0到3，x[n] = [1,2,3,4]
n_original = np.arange(0, 4)
x = np.array([1, 2, 3, 4])

# 2. 尺度变换：k=2展宽（y[n] = x[n/2]，非2的倍数补0）
k = 2
# 展宽后的n范围：0到k*(len(n_original)-1)（确保覆盖所有有效点）
n_expand = np.arange(0, k*(len(n_original)-1) + 1)
y = np.zeros_like(n_expand, dtype=int)  # 初始化全0

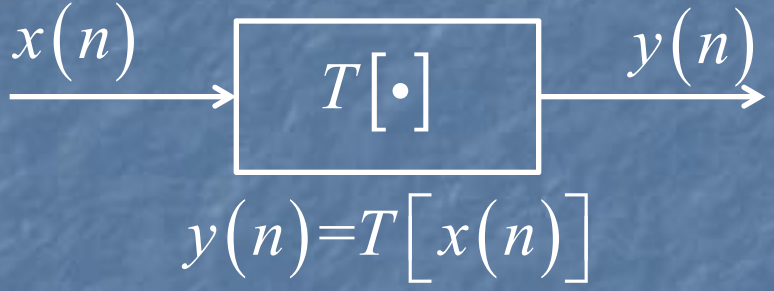
# 仅当n是k的倍数时，赋值x[n/k]
for i, n in enumerate(n_expand):
    if n % k == 0:  # n是k的倍数，n/k为整数
        idx = n // k  # 对应原始序列的索引
        if idx < len(x):  # 确保索引在原始序列范围内
            y[i] = x[idx]

# 画图对比：展宽后多了补零的点
plt.subplot(2,1,1)
plt.stem(n_original, x, label='原始序列 x[n]')
plt.legend()
plt.subplot(2,1,2)
plt.stem(n_expand, y, label=f'k=2展宽序列 y[n] = x[n/2]')
plt.legend()
plt.show()
```



1.3 时域离散系统

时域离散系统:对输入序列做运算，输出新序列的处理器



$$y(n)=T\left[x(n)\right]$$

比如：

y [n] = 2x [n]（放大 2 倍）

y [n] = x [n] + x [n-1]（当前输入 + 前一时刻输入）

y [n] = x [2n]（压缩）

1.线性系统

$y_1(n)=T[x_1(n)]$　$y_2(n)=T[x_2(n)]$

同时满足齐次性和可加性的系统称为线性系统。

“可加性”（输入 1 + 输入 2 的输出 = 输入 1 的输出 + 输入 2 的输出）

$$T[x_1(n)+x_2(n)]=y_1(n)+y_2(n)$$

“齐次性”（输入放大 a 倍，输出也放大 a倍）

$$T[ax_1(n)]=ay_1(n)$$

线性系统的最大价值是“可分解性”—— 复杂输入（如语音、图像）可拆成多个简单输入（如正弦信号）的叠加，分别处理后再叠加，结果和直接处理复杂输入一致，大幅降低处理难度。

$$y(n)=T[ax_1(n)+bx_2(n)]=ay_1(n)+by_2(n)$$

例. 判断下列系统是否为线性系统？

1）$y(n)=ax(n)+b$　　2）$y(n)=x(n)\sin(\frac{2\pi}{9}n+\frac{\pi}{7})$

非线性系统　　线性系统

求（1）　$y_1(n)=T[x_1(n)]$　　$y_2(n)=T[x_2(n)]$

（2）　$T[ax_1(n)+bx_2(n)]$

（3）　$ay_1(n)+by_2(n)?T[ax_1(n)+bx_2(n)]$

$y_1(n)=T[x_1(n)]=ax_1(n)+b$　　$y_2(n)=T[x_2(n)]=ax_2(n)+b$

$T[ax_1(n)+bx_2(n)]=a[ax_1(n)+bx_2(n)]+b=a^2x_1(n)+abx_2(n)+b$

$ay_1(n)+by_2(n)=a^2x_1(n)+ab+abx_2(n)+b^2$

代码演示：

linear_system_check.py

```python
import numpy as np

# 定义两个测试输入序列
n = np.arange(0, 5)
x1 = np.array([1, 2, 3, 4, 5])  # 输入1
x2 = np.array([2, 3, 4, 5, 6])  # 输入2
a, b = 2, 3  # 任意常数

# ----------------------测试1：线性系统 y[n] = 2x[n] + x[n-1]（x[-1]补0）-------------
def linear_system(x, n):
    y = np.zeros_like(x)
    for i in range(len(n)):
        if i == 0:
            x_prev = 0  # x[n-1]在n=0时补0
        else:
            x_prev = x[i-1]
        y[i] = 2 * x[i] + x_prev
    return y

# 计算1：先叠加输入，再过系统 T(a x1 + b x2)
x_combined = a * x1 + b * x2
y_combined = linear_system(x_combined, n)

# 计算2：先分别过系统，再叠加 a T(x1) + b T(x2)
y1 = linear_system(x1, n)
y2 = linear_system(x2, n)
y_separate = a * y1 + b * y2

# 验证是否相等（允许浮点误差）
is_linear = np.allclose(y_combined, y_separate)
print("系统1（y=2x[n]+x[n-1]）是否线性？", is_linear)  # 输出True

# ----------------------测试2：非线性系统 y[n] = x[n]²----------------------
def nonlinear_system(x):
    return x ** 2

# 计算1：先叠加输入，再过系统 T(a x1 + b x2)
y_combined_non = nonlinear_system(a * x1 + b * x2)

# 计算2：先分别过系统，再叠加 a T(x1) + b T(x2)
y1_non = nonlinear_system(x1)
y2_non = nonlinear_system(x2)
y_separate_non = a * y1_non + b * y2_non

# 验证是否相等
is_linear_non = np.allclose(y_combined_non, y_separate_non)
print("系统2（y=x[n]²）是否线性？", is_linear_non)  # 输出False
```

- 系统1（$y=2x[n]+x[n-1]$）是否线性？ True
- 系统2（$y=x[n]^2$）是否线性？ False

例： 证明 y(n)=3x(n)+4是非线性系统

证： 已知 $y(n)=T[x(n)]=3x(n)+4$

(1) $y_1(n)=T[x_1(n)]=3x_1(n)+4$

$y_2(n)=T[x_2(n)]=3x_2(n)+4$

$ay_1(n)+by_2(n)=3ax_1(n)+3bx_2(n)+4a+4b$

(2) $T[ax_1(n)+bx_2(n)]=3[ax_1(n)+bx_2(n)]+4$

(3) $T[ax_1(n)+bx_2(n)]\neq ay_1(n)+by_2(n)$

所以，该系统是非线性系统

- 如果已知某个系统是线性系统，则满足

$$T[a_1x_1(n)+a_2x_2(n)]=T\left[a_1x_1(n)\right]+T\left[a_2x_2(n)\right]$$
$$=a_1T\left[x_1(n)\right]+a_2T\left[x_2(n)\right]$$
$$=a_1y_1(n)+a_2y_2(n)$$

$$T[\sum_{k=1}^{N}a_kx_k(n)]=\sum_{k=1}^{N}T\left[a_kx_k(n)\right]$$
$$=\sum_{k=1}^{N}a_kT\left[x_k(n)\right]=\sum_{k=1}^{N}a_ky_k(n)$$

线性系统的输入为一个序列的和，其输出等效为序列每个元素的系统输出求和。

2. 时不变系统 $y(n)=T[x(n)]$

若系统对输入信号的响应与信号加于系统的时刻无关，则称为时不变系统。

时不变系统的核心是“输入时移 k 位，输出也时移 k 位”

$$y(n-n_0)=T[x(n-n_0)]$$

工程场景：系统的处理能力不随时间变化

比如今天用滤波器处理语音信号，和明天用同一个滤波器处理相同的语音信号，结果完全一致。

例. 判断下列系统是否为时不变系统？

判断方法：验证 $T[x(n-n_0)] \overset{?}{=} y(n-n_0)$

1）$y(n)=ax(n)+b$

时不变系统

2）$y(n)=x(n)\sin(\frac{2\pi}{9}n+\frac{\pi}{7})$

时变系统

exercise

- 1. 对于系统 $y(n)=(n+1)x(n)$ ，

  因为 $y(n-n_0)\neq T[x(n-n_0)]$ 因此该系统属于时变系统。（正确）

exercise

- 2. 对于系统 $y(n)=n^2x(n)$ ，

  因为 $y(n-n_0)=T[x(n-n_0)]=(n-n_0)^2x(n-n_0)$ 因此该系统属于时不变系统。（错误）

- 既满足线性和时不变特性的系统称为线性时不变系统(Linear Time-Invariant，LTI)

- 本书只涉及LTI系统

3. 因果系统

- 若系统 n时刻的输出，只取决于n时刻以及n时刻以前的输入序列，而与n时刻以后的输入无关，则称该系统为因果系统。

- 线性时不变系统是因果系统的充要条件：

系统的单位脉冲响应：　$h(n)=0 \quad n<0$

系统的单位脉冲响应？

设系统的输入$x(n)=\delta(n)$时，系统输出初始状态为零，定义这种条件下系统输出为单位脉冲相应$h(n)$。

零状态响应：0时刻以前响应为0，响应取决于当前信号

零输入响应： 0时刻没有信号，响应取决于0时刻以前的初始储能

完全响应：零状态响应+零输入响应

零状态响应

$$\delta(n) \rightarrow h(n)$$

工程场景：实时处理 vs 离线处理

实时场景（必须因果）：耳机降噪（实时处理当前环境噪声）、雷达实时目标检测（处理当前回波信号）。

离线场景（可非因果）：视频后期剪辑（可调用未来帧做特效）、历史气象数据复盘（可用未来数据校准过去）。

4. 稳定系统

- 稳定系统是有界输入产生有界输出(BIBO)的系统

若 $|x(n)| \le M < +\infty$

则 $|y(n)| \le P < +\infty$

- 线性时不变系统是稳定系统的充要条件：

系统单位脉冲响应绝对可和： $\sum_{n=-\infty}^{\infty} |h(n)| < +\infty$

工程场景：稳定是系统的“安全底线”

如果系统不稳定，即使输入很小（如微小噪声），输出也可能无限增大（如放大器自激、电机转速失控），导致设备损坏甚至危险。

所有“实际运行的电子设备”都是稳定系统

比如手机的信号放大器、音响的功率放大器、工业电机的控制器，都必须满足 BIBO 稳定 —— 否则输入微小波动（如电源噪声），输出就会失控，设备无法正常工作。

例: 某线性时不变系统，其单位脉冲响应为

$$h(n)=a^n u(-n)$$

试讨论其是否是因果的、稳定的。

分析：1、因果性—— $h(n)=0 \quad n<0$

2、稳定性——是否$h(n)$绝对可和？

解：1）讨论因果性： Q $n<0$时 $h(n)\neq 0$

$\therefore$该系统是非因果系统

2）讨论稳定性：

$$Q\ \sum_{n=-\infty}^{\infty}|h(n)|=\sum_{n=-\infty}^{0}|a^{n}|=\sum_{n=0}^{\infty}|a|^{-n}$$

$$=\begin{cases}\dfrac{1}{1-|a|^{-1}} & |a|>1\\ \infty & |a|\le 1\end{cases}$$

$\therefore$ 当$|a|>1$时系统稳定，当$|a|\le 1$时系统不稳定

等比数列求和：

$$S_n = n \times a_1 \quad (q=1)$$

$$S_n = a_1 \cdot \frac{1-q^n}{1-q} = \frac{a_1 - a_n \cdot q}{1-q} \quad (q \neq 1)$$

例: 某线性时不变系统，其单位脉冲响应为

$$h(n)=u(n)$$

试讨论其因果性、稳定性

解：1) 因果性: $h(n)=0 \quad n<0$

2) 稳定性: $\sum_{n=-\infty}^{\infty}|h(n)|=\sum_{n=0}^{\infty}|u(n)|=\infty$

结论：该系统是因果、不稳定的

- 工程中最常用的是"**线性时不变因果稳定系统**"—— 线性保证可分解处理，时不变保证性能稳定，因果保证实时工作，稳定保证设备安全"，这也是后续学习 "卷积、傅里叶变换、滤波器设计" 的前提（所有这些工具都只适用于线性时不变系统）。

5. 线性卷积(convolution)

- 任意离散信号 $x(n)$ 可以表示为单位脉冲序列 $\delta(n)$ 的线性组合（移位加权和）。

$$x(n)=\sum_{k=-\infty}^{\infty}x(k)\delta(n-k)$$

- 线性时不变离散系统在输入为 $\delta(n)$ 时的零状态响应为单位脉冲响应 $h(n)$，即

$$h(n)=T[\delta(n)]$$

$$x(n)=\sum_{k=-\infty}^{\infty}x(k)\delta(n-k)$$

$$y(n)=T[x(n)]=T\left[\sum_{k=-\infty}^{\infty}x(k)\delta(n-k)\right]$$

与k相关的常数

由叠加性有

$$y(n)=T[x(n)]=\sum_{k=-\infty}^{\infty}x(k)T[\delta(n-k)] \qquad h(n)=T[\delta(n)]$$

根据时不变性质 $h(n-k)=T[\delta(n-k)]$

$$y(n)=T[x(n)]=\sum_{k=-\infty}^{\infty}x(k)h(n-k)=x(n)*h(n)$$

零状态响应

$$y(n)=x(n)*h(n)=\sum_{k=-\infty}^{\infty}x(k)h(n-k)$$

工程场景：回声的“信号叠加”

系统：房间的回声系统（单位脉冲响应 h [n] = [1, 0.5, 0.2]，表示“直达声 + 1 次回声 + 2 次回声”）

输入 x [n]：短促的拍手声（比如 x [n] = [1,0,0,...]，即单位脉冲）

卷积结果 y [n] = x [n]*h [n] = [1, 0.5, 0.2, 0,...]，刚好是“拍手声 + 0.5 倍回声 + 0.2 倍回声”，完美模拟实际回声效果

卷积的两个核心作用

- 系统输出计算：对任意 LTI 系统，只要知道单位脉冲响应 h [n]，就能通过 x [n]*h [n] 得到任意输入 x [n] 的输出。

- 信号合成 / 滤波：通过设计不同的 h [n]（如平滑的 h [n] 实现低通滤波，尖锐的 h [n] 实现边缘检测），可以用卷积“定制”输出信号的形态，这是数字信号处理的核心思想。

如何计算线性卷积？

- 图解法
- 列表法
- 解析法
- MATLAB法

图解法

“滑动叠加”

例. 设 $x(n)=R_4(n)$，$h(n)=R_4(n)$

求 $y(n)=x(n)*h(n)$

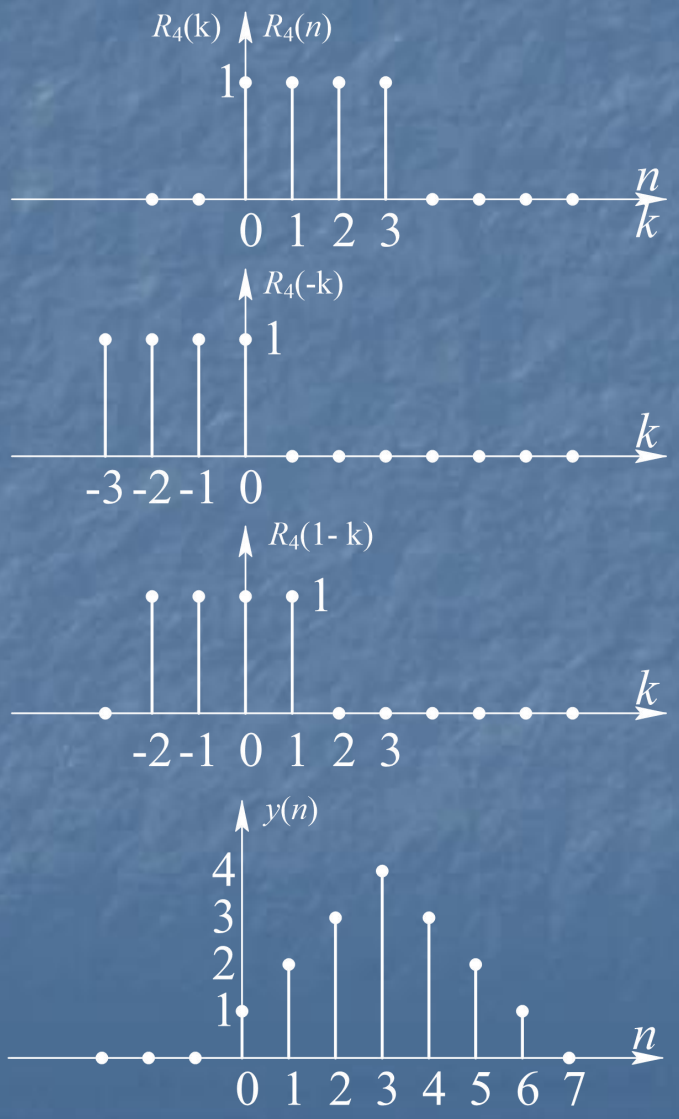
解. $y(n)=\sum_{k=-\infty}^{\infty}R_4(k)R_4(n-k)$

1）置换：$n\to k$

2）反折：$R_4(k)\to R_4(-k)$

3）移位：$R_4(-k)\to R_4(n-k)$

4）相乘：如 $n=1$ 时，

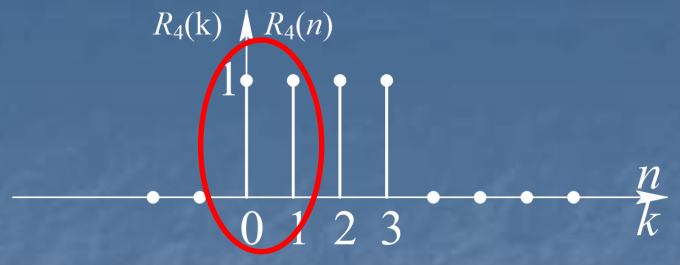


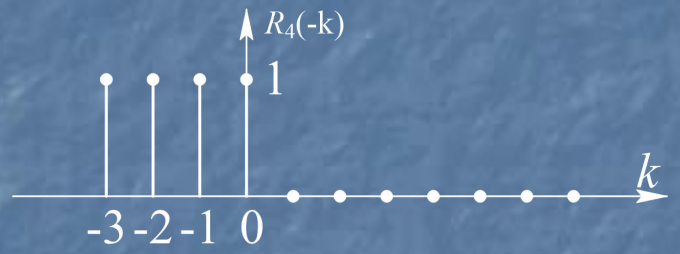
$$y(1)=\sum_{k=-\infty}^{\infty}R_4(k)R_4(1-k)$$

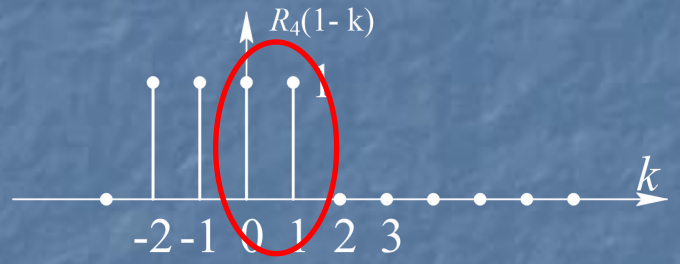
$$=\sum_{k=0}^{1}R_4(k)R_4(1-k)$$

$$=R_4(0)R_4(1)+R_4(1)R_4(0)$$

$$=1+1=2$$

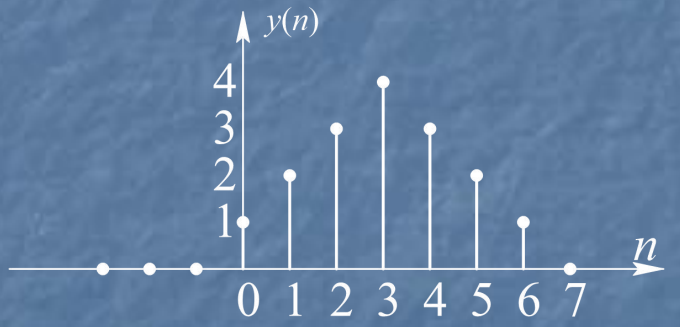






5）求和式 n=[0,6]

y(n)={1,2,3,4,3,2,1}



列表法

$x(m)$				1	1	1	1				
$h(m)$				1	1	1	1				
$h(-m)$	1	1	1	1							$y(0)=1$
$h(1-m)$		1	1	1	1						$y(1)=2$
$h(2-m)$			1	1	1	1					$y(2)=3$
$h(3-m)$				1	1	1	1				$y(3)=4$
$h(4-m)$					1	1	1	1			$y(4)=3$
$h(5-m)$						1	1	1	1		$y(5)=2$
$h(6-m)$							1	1	1	1	$y(6)=1$

◆解析法

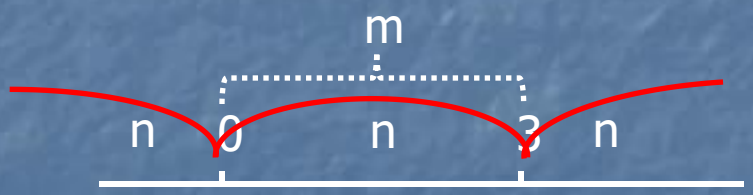
如果已知两个卷积信号的解析表达式，则可以直接按照卷积式进行计算，下面举例说明。

设$x(n)=a^n u(n)$，$h(n)=R_4(n)$，求$y(n)=x(n)*h(n)$。

$$y(n)=h(n)*x(n)=\sum_{m=-\infty}^{\infty}R_4(m)a^{n-m}u(n-m)$$

$R_4(m)$　当$0\le m\le 3$时，取非零值

$u(n-m)$　当$m\le n$时，取非零值



$n<0$时　$y(n)=0$

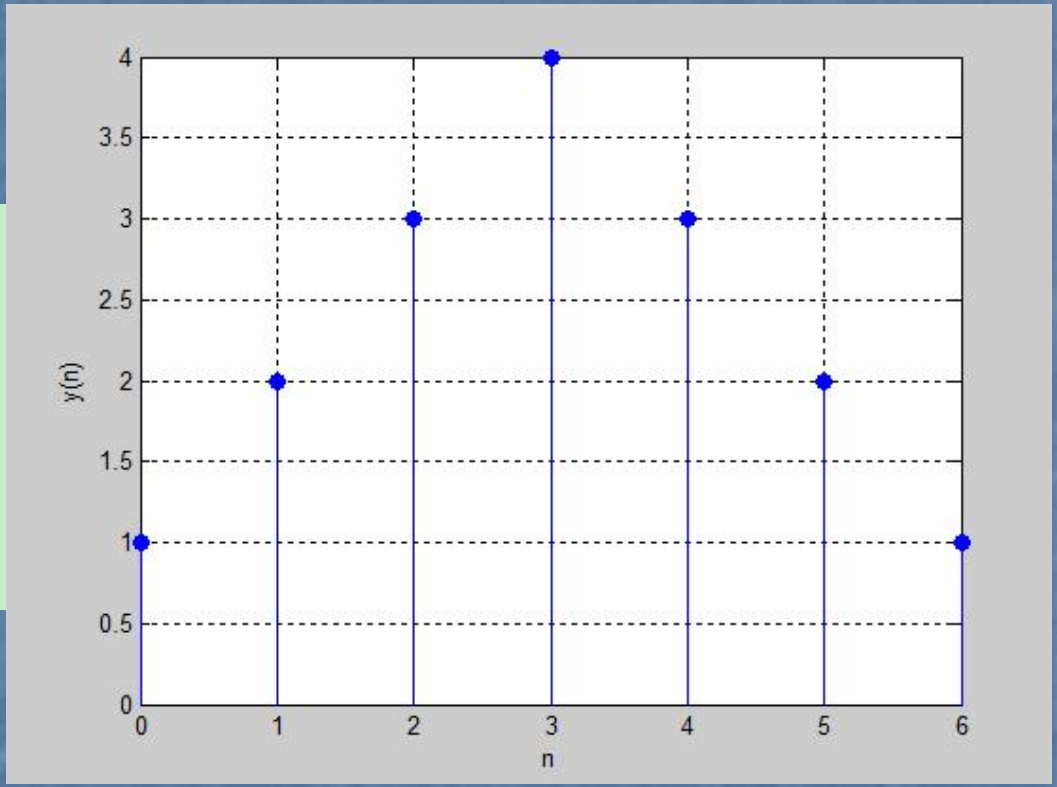
$0\le n\le 3$时　$y(n)=\sum_{m=0}^{n}a^{n-m}=a^n\dfrac{1-a^{-n-1}}{1-a^{-1}}$

$n\ge 4$　$y(n)=\sum_{m=0}^{3}a^{n-m}=a^n\dfrac{1-a^{-4}}{1-a^{-1}}$

MATLAB法

卷积函数C=conv(A,B)

```
a=[1 1 1 1];
b=[1 1 1 1];
c=conv(a,b); %卷积 convolution
M=length(c)-1; %序列从0开始
n=0:1:M;
stem(n,c,'fill'); %在n的位置画出c的值，格式为‘fill’
grid on;  %添加网格线
xlabel('n');ylabel('y(n)');%x，y轴标签
```



线性卷积服从交换律、结合律和分配律。它们分别用公式表示如下：

$$x(n) * h(n) = h(n) * x(n)$$

$$x(n) * [h_1(n) * h_2(n)] = [x(n) * h_1(n)] * h_2(n)$$

$$x(n) * [h_1(n) + h_2(n)] = x(n) * h_1(n) + x(n) * h_2(n)$$

若有限长序列x(n)的长度为N，h(n)的长度为M，则其卷积和的长度L为

L=N+M-1

1.4线性常系数差分方程

- 一个N阶线性常系数差分方程表示为：

$$\sum_{i=0}^{N} a_i y(n-i) = \sum_{i=0}^{M} b_i x(n-i)$$

其中：$a_0 = 1$，a_i，b_i是常数

特点：1）常系数　2）无交叉项

3）阶数，y(n-i)中i的取值最大最小之差

- 求解差分方程的方法：

1）时域方法：经典解法，递推解法

2）变换域方法　3）MATLAB法

例. 若描述某离散时间LTI系统的差分方程为

$$y(n)+3y(n-1)+2y(n-2)=x(n)$$

求系统的单位脉冲响应 $h(n)$。分析：输入$\delta(n)$系统零状态响应

解: $h(n)$ 满足方程

$$h(n)=\delta(n)-3h(n-1)-2h(n-2)$$

零状态响应有 $h(-1)=h(-2)=0$ ，因此

$$h(0)=\delta(0)-3h(-1)-2h(-2)=1$$

$$h(1)=\delta(1)-3h(0)-2h(-1)=-3$$

$$h(2)=\delta(2)-3h(1)-2h(0)=7$$

$$h(3)=\delta(3)-3h(2)-2h(1)=-15$$

……

$$h(n)=-(-1)^n+2(-2)^n \quad n\geq 0$$

◆MATLAB法

解差分方程yn=filter(B,A,xn,xi)

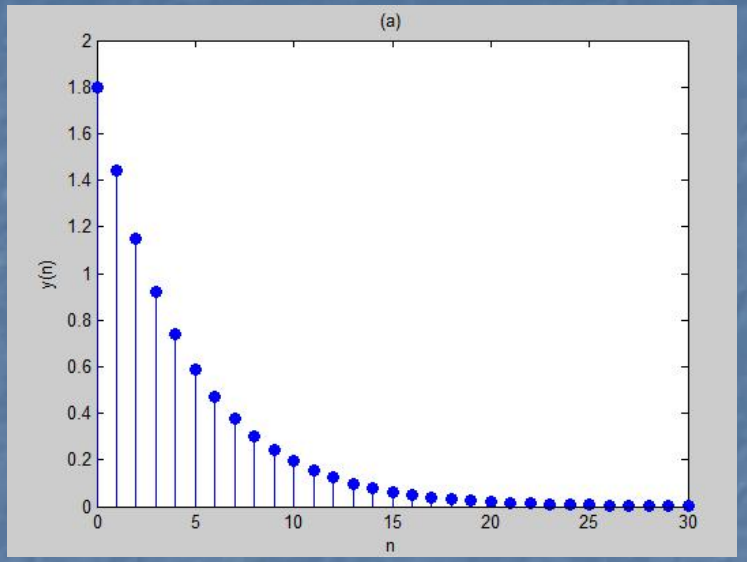
xi=filtic(B,A,ys)

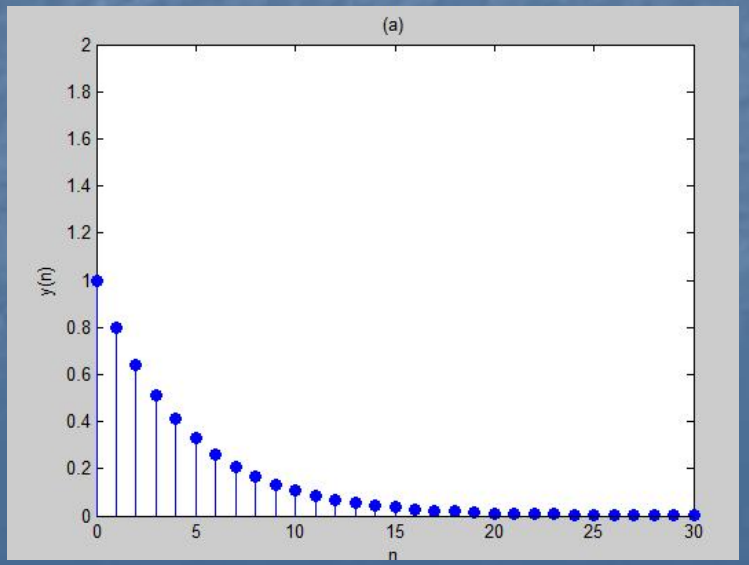
$$y(n)-ay(n-1)=x(n)$$

```
a=0.8;  ys=1;   %设差分方程系数a=0.8, %初始状态: y(-1)=1
xn=[1, zeros(1, 30)];   %x(n)=单位脉冲序列, 长度N=31
B=1; A=[1, -a];         %差分方程系数
xi=filtic(B, A, ys);    %由初始条件计算等效初始条件的输入序列xi
yn=filter(B, A, xn, xi);%调用filter解差分方程, 求系统输出信号y(n)
M=length(yn)-1;
n=0:1:M;
stem(n,yn,'fill');
title('(a)'); xlabel('n'); ylabel('y(n)')
axis([0,30,0,2]);%坐标轴范围
```

```
a=0.8;  ys=0;   %设差分方程系数a=0.8, %初始状态: y(-1)=1
xn=[1, zeros(1, 30)];   %x(n)=单位脉冲序列, 长度N=31
B=1; A=[1, -a];         %差分方程系数
xi=filtic(B, A, ys);    %由初始条件计算等效初始条件的输入序列xi
yn=filter(B, A, xn, xi);%调用filter解差分方程, 求系统输出信号y(n)
M=length(yn)-1;
n=0:1:M;
stem(n,yn,'fill');
title('(a)'); xlabel('n'); ylabel('y(n)')
axis([0,30,0,2]);%坐标轴范围
```

初始条件输入序列





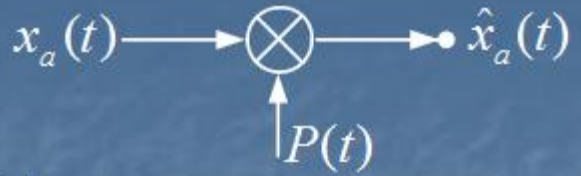
1.5 采样定理(Sampling Theorem)

模拟信号(Analog Signal) → 数字信号(Digital Signal)



数字信号处理系统简单框图

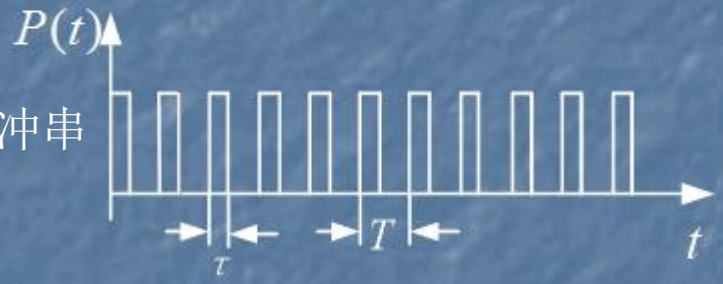


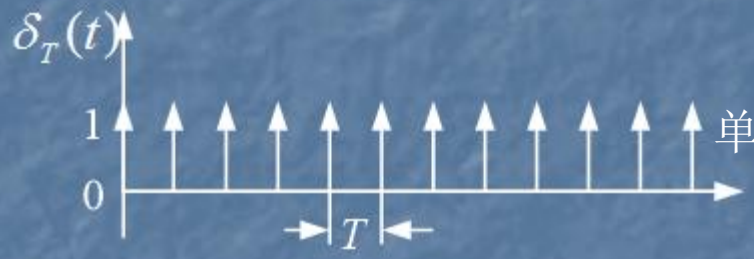






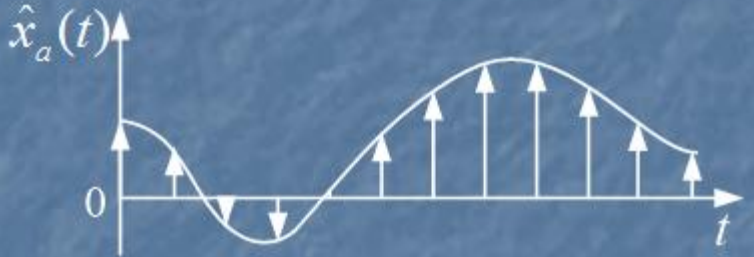
矩形脉冲串





单位冲激串





$$x_a(t) \rightarrow \hat{x}_a(t)$$

$$\hat{x}_a(t) = x_a(t)P(t)$$

当$\tau \rightarrow 0$ 理想采样

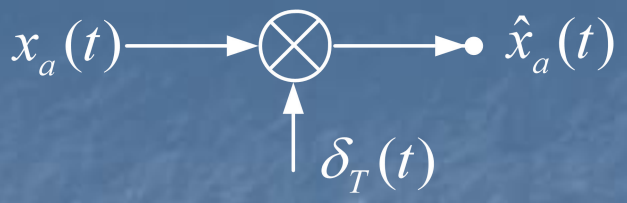
$$\hat{x}_a(t) = x_a(t)\delta_T(t) = \sum_{n=-\infty}^{\infty} x_a(\mathrm{t})\delta(t\text{-}\mathrm{nT})$$

$$= \sum_{n=-\infty}^{\infty} x_a(\mathrm{nT})\delta(t\text{-}\mathrm{nT})$$

问题：

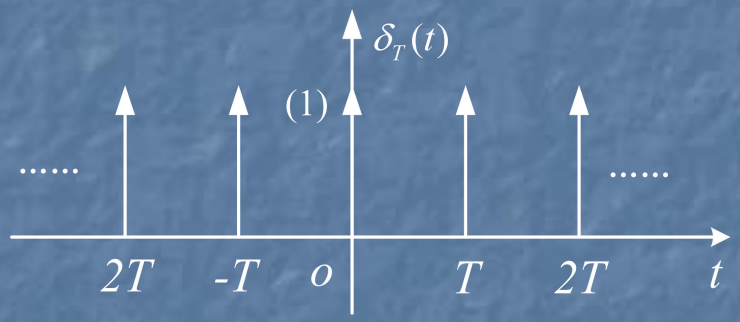
- 抽样后离散信号的频谱(Spectrum)是什么样的？它与抽样前的信号频谱有什么关系？
- 抽样信号是否保留了原信号的所有信息？即在什么条件下，可以从抽样信号中无失真地恢复出原始信号？

1.以理想抽样为例进行分析



抽样脉冲为周期冲激函数：

$$\delta_T(t)=\sum_{n=-\infty}^{\infty}\delta(t-nT)$$



抽样信号 $\hat{x}_a(t)$：

$$\hat{x}_a(t)=x_a(t)\cdot\delta_T(t)=\sum_{n=-\infty}^{\infty}x_a(nT)\delta(t-nT)$$

抽样信号$\hat{x}_a(t)$的频谱$\hat{X}_a(j\Omega)$？

$$\hat{x}_a(t) = x_a(t) \cdot \delta_T(t)$$ 由频域卷积定理 P41（43）

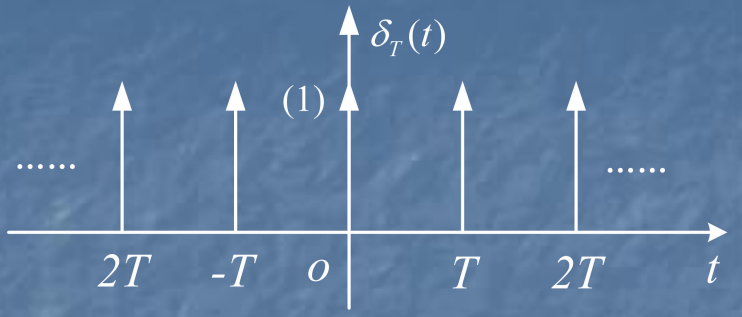
$$\hat{X}_a(j\Omega) = \frac{1}{2\pi} X_a(j\Omega) * P_\delta(j\Omega)$$ 冲激函数线性组合的FT

$$P_\delta(j\Omega) = \sum_{k=-\infty}^{\infty} 2\pi a_k \delta(\Omega - k\Omega_s)$$

$$\Omega_s = \frac{2\pi}{T}$$

$$P_\delta(j\Omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\Omega-k\Omega_s)?$$

周期冲激函数：$\delta_T(t)=\sum_{n=-\infty}^{\infty}\delta(t-nT)$



周期信号的傅里叶级数：$x(t)=\sum_{k=-\infty}^{\infty}a_k e^{jk(2\pi/T)t}$

假设某个信号x(t)傅里叶变换：$X(j\omega)=2\pi\delta(\omega-\omega_0)$

傅里叶反变换：$x(t)=\frac{1}{2\pi}\int_{-\infty}^{\infty}2\pi\delta(\omega-\omega_0)e^{j\omega t}d\omega=e^{j\omega_0 t}$

推广到冲激函数线性组合：$X(j\omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\omega-k\omega_0)$

傅里叶反变换：

$$x(t)=\frac{1}{2\pi}\int_{-\infty}^{\infty}\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\omega-k\omega_0)e^{j\omega t}d\omega=\sum_{k=-\infty}^{\infty}a_k e^{jk\omega_0 t}$$

周期信号的傅里叶变换表示

周期信号的傅里叶级数： $x(t)=\sum_{k=-\infty}^{\infty}a_k e^{jk(2\pi/T)t}$

频域构造傅里叶变换： $X(j\omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\omega-k\omega_0)$

$$X(j\omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\omega-k\omega_0) \underset{\text{傅里叶变换}}{\overset{\text{傅里叶反变换}}{\rightleftharpoons}} x(t)=\sum_{k=-\infty}^{\infty}a_k e^{jk\omega_0 t}$$

周期信号FT $\longrightarrow \sum_{k=-\infty}^{\infty}a_k e^{jk(2\pi/T)t} \longrightarrow \sum_{k=-\infty}^{\infty}2\pi a_k\delta(\omega-k\omega_0)$

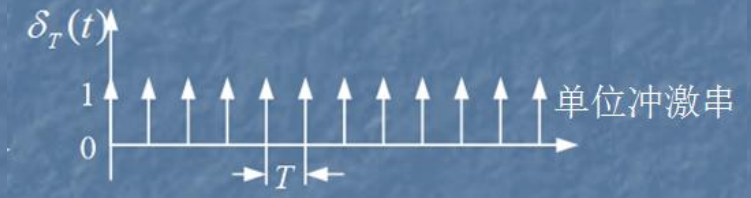
$$a_k=\frac{1}{T}\int_T x(t)e^{-jk\omega_0 t}dt$$

抽样信号$\hat{x}_a(t)$的频谱$\hat{X}_a(j\Omega)$？

$$\hat{x}_a(t)=x_a(t)\cdot\delta_T(t)$$ 由频域卷积定理 P41

$$\hat{X}_a(j\Omega)=\frac{1}{2\pi}X_a(j\Omega)*P_\delta(j\Omega)$$ 冲激函数线性组合的FT

$$P_\delta(j\Omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\Omega-k\Omega_s)$$



$$a_k=\frac{1}{T}\int_{-T/2}^{T/2}\delta_T(t)e^{-jk\Omega_s t}dt$$ 在[-T/2, T/2]内，$\delta_T(t)=\delta(t)$

$$a_k=\frac{1}{T}\int_{-T/2}^{T/2}\delta(t)e^{-jk\Omega_s t}dt=\frac{1}{T}$$

$$P_\delta(j\Omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\Omega-k\Omega_s)=\frac{2\pi}{T}\sum_{k=-\infty}^{\infty}\delta(\Omega-k\Omega_s)$$

$$\hat{X}_a(j\Omega)=\frac{1}{2\pi}X_a(j\Omega)*P_\delta(j\Omega)$$

$$P_\delta(j\Omega)=\sum_{k=-\infty}^{\infty}2\pi a_k\delta(\Omega-k\Omega_s)=\frac{2\pi}{T}\sum_{k=-\infty}^{\infty}\delta(\Omega-k\Omega_s)$$

$$\hat{X}_a(j\Omega)=\frac{1}{2\pi}[X_a(j\Omega)*\frac{2\pi}{T}\sum_{k=-\infty}^{\infty}\delta(\Omega-k\Omega_s)]$$

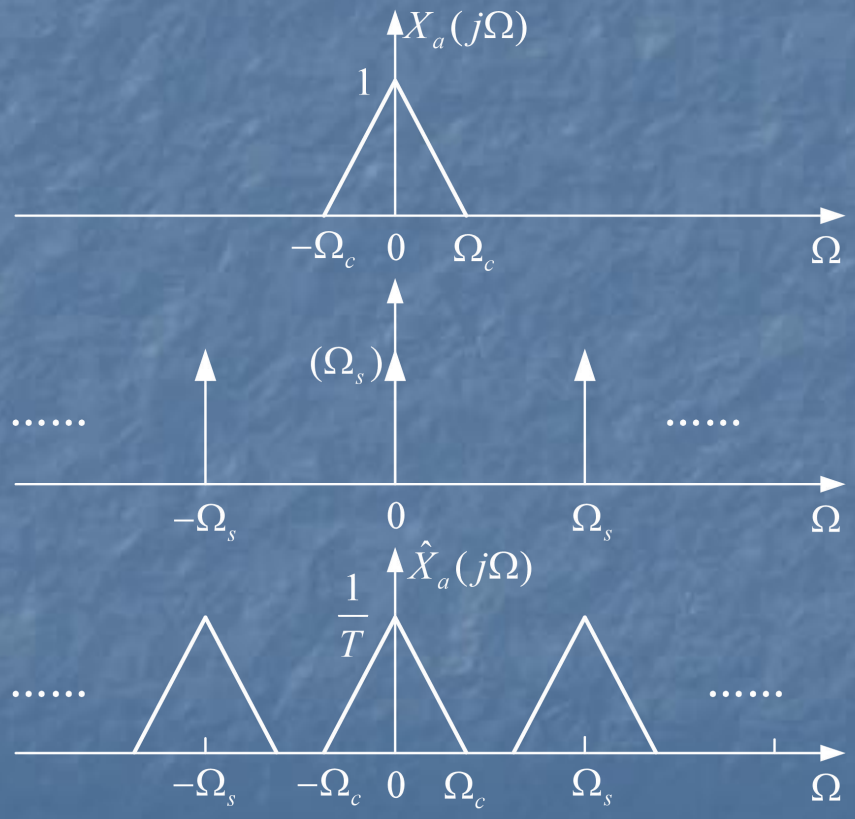
$$=\frac{1}{T}[X_a(j\Omega)*\sum_{k=-\infty}^{\infty}\delta(\Omega-k\Omega_s)]$$

$$=\frac{1}{T}\int_{-\infty}^{\infty}X_a(j\theta)\sum_{k=-\infty}^{\infty}\delta(\Omega-k\Omega_s-\theta)d\theta$$

$$=\frac{1}{T}\sum_{k=-\infty}^{\infty}\int_{-\infty}^{\infty}X_a(j\theta)\delta(\Omega-k\Omega_s-\theta)d\theta$$

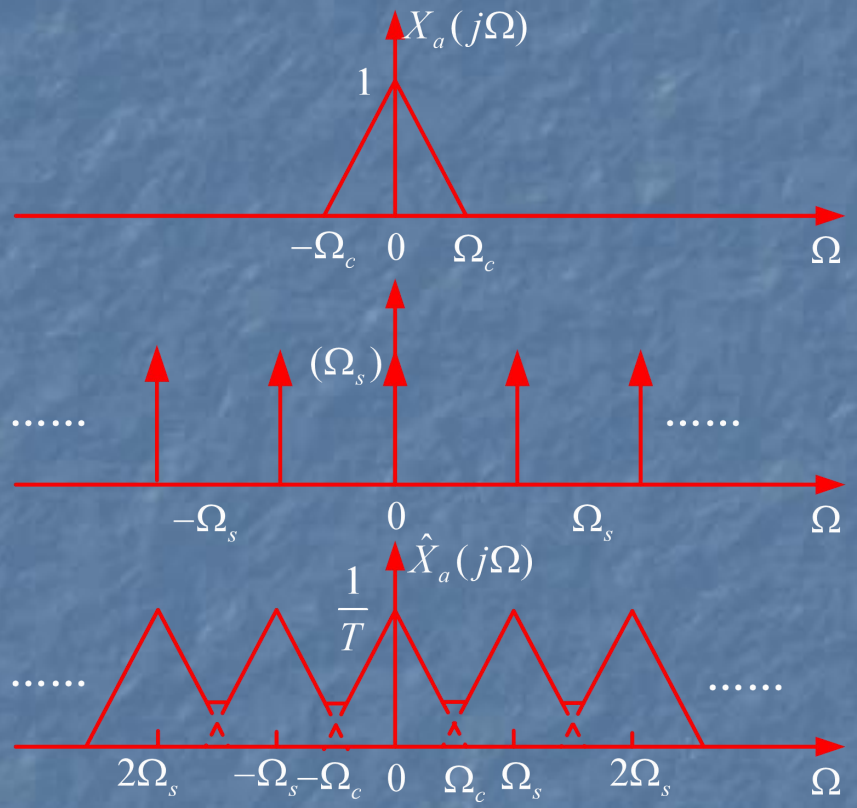
$$=\frac{1}{T}\sum_{k=-\infty}^{\infty}X_a(j\Omega-jk\Omega_s)$$

$$\hat{X}_a(j\Omega)=\frac{1}{T}[X_a(j\Omega)*\sum_{k=-\infty}^{\infty}\delta(\Omega-k\Omega_s)]=\frac{1}{T}\sum_{k=-\infty}^{\infty}X_a(j\Omega-jk\Omega_s)$$



Ω_c 最高频率

Ω_s 采样角频率



频谱混叠(Aliasing)

◆奈奎斯特抽样定理：

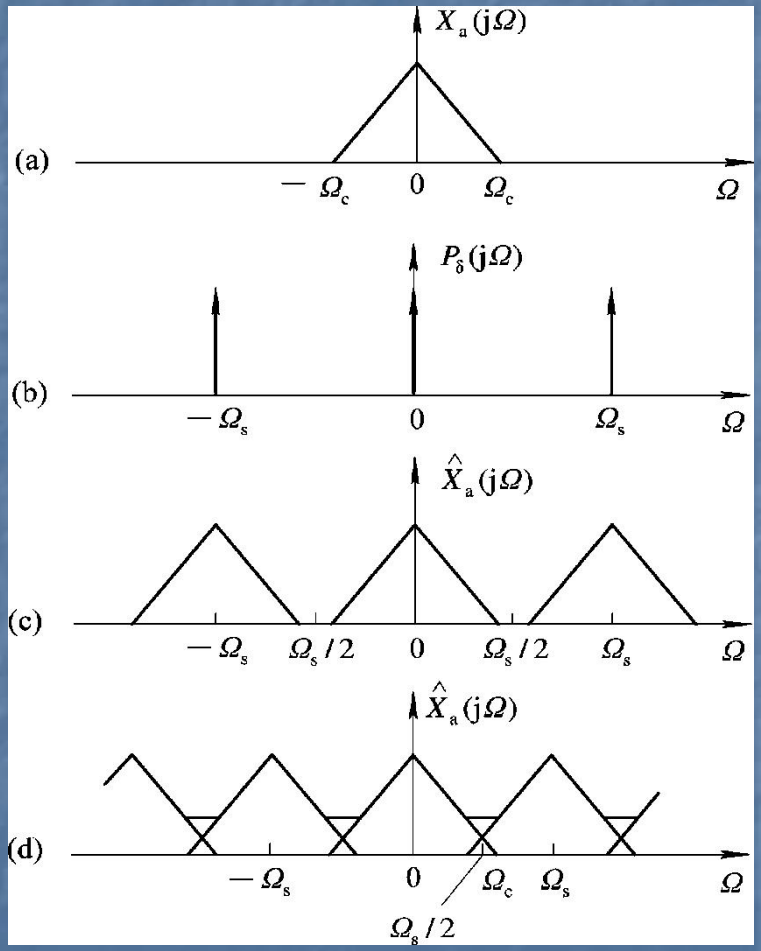
(1)带限信号

$$X_a(j\Omega)=0 \quad |\Omega|>\Omega_c$$

(2) $\Omega_s \geq 2\Omega_c$ $\Omega_s=\dfrac{2\pi}{T}=F_s\cdot 2\pi$

$F_s \geq 2f_c$ 或 $T_s \leq \dfrac{1}{2f_c}$

折叠频率： $F_s/2$



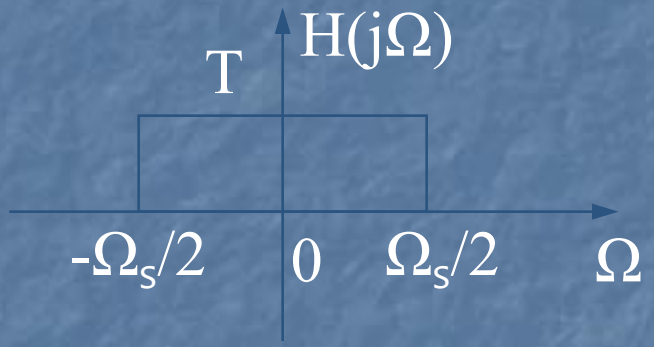
2. 信号的恢复　　$\hat{x}_a(t) \longrightarrow x_a(t)$

A、频域上恢复　$\hat{X}_a(j\Omega) \longrightarrow X_a(j\Omega)$

利用截止频率为$\Omega_s/2$的理想低通滤波器可以从抽样信号中不失真地恢复原模拟信号。

理想低通滤波器:

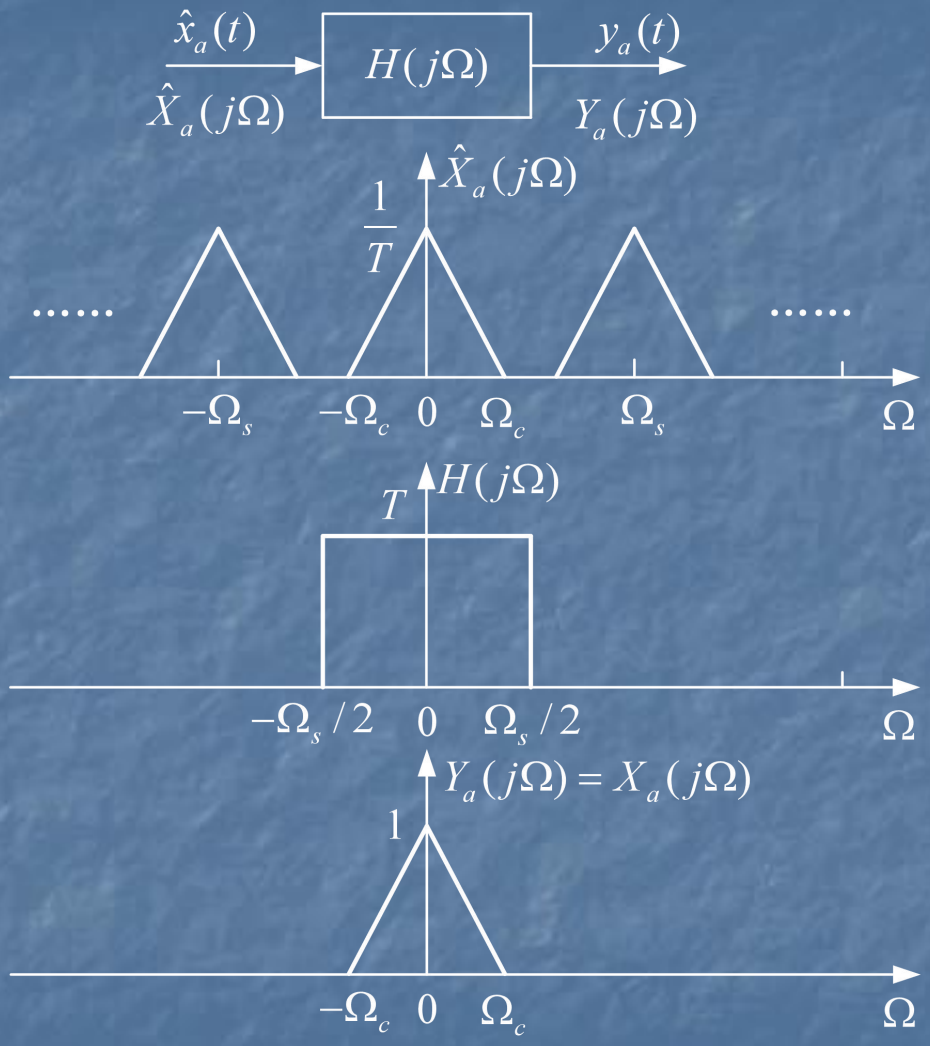
$$H(j\Omega)=\begin{cases} T & |\Omega| \le \Omega_s/2 \\ 0 & |\Omega| > \Omega_s/2 \end{cases}$$





$$Y_a(j\Omega)=\hat{X}_a(j\Omega)\cdot H(j\Omega)=X_a(j\Omega)$$

频域


$\hat{x}_a(t)$
$\hat{X}_a(j\Omega)$
$H(j\Omega)$
$y_a(t)$
$Y_a(j\Omega)$
$\hat{X}_a(j\Omega)$
$\frac{1}{T}$
$-\Omega_s$
$-\Omega_c$
0
Ω_c
Ω_s
Ω
$H(j\Omega)$
T
$-\Omega_s/2$
0
$\Omega_s/2$
Ω
$Y_a(j\Omega)=X_a(j\Omega)$
1
$-\Omega_c$
0
Ω_c
Ω

B、时域恢复 $X_a(j\Omega) \longrightarrow x_a(t)$

$$X_a(j\Omega) = \hat{X}_a(j\Omega) \cdot H(j\Omega)$$ 由时域卷积定理

$$x_a(t) = \hat{x}_a(t) * h(t)$$

$$H(j\Omega) = \begin{cases} T & |\Omega| \le \Omega_s/2 \\ 0 & |\Omega| > \Omega_s/2 \end{cases}$$ 由傅里叶反变换

$$h(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} H(j\Omega)e^{j\Omega t}d\Omega$$

$$= \frac{1}{2\pi}\int_{-\Omega_s/2}^{\Omega_s/2} Te^{j\Omega t}d\Omega = \frac{\sin(\Omega_s t/2)}{\Omega_s t/2} \qquad \Omega_s = \frac{2\pi}{T}$$

$$= \frac{\sin(\pi t/T)}{\pi t/T}$$

时域恢复 $X_a(j\Omega) \longrightarrow x_a(t)$

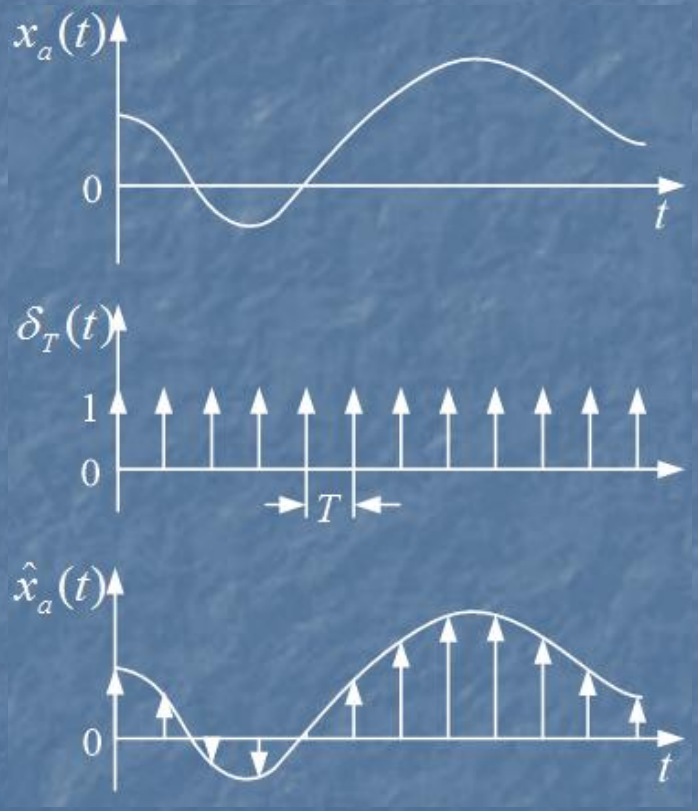
$$X_a(j\Omega) = \hat{X}_a(j\Omega) \cdot H(j\Omega)$$

由时域卷积定理

$$x_a(t) = \hat{x}_a(t) * h(t)$$

$$x_a(t) = \int_{-\infty}^{\infty} \hat{x}_a(\tau) h(t-\tau) d\tau$$

$$\hat{x}_a(t) = \sum_{n=-\infty}^{\infty} x_a(nT)\delta(t-nT)$$



时域恢复 $X_a(j\Omega) \longrightarrow x_a(t)$

$$x_a(t)=\int_{-\infty}^{\infty}\hat{x}_a(\tau)h(t-\tau)d\tau \qquad \hat{x}_a(t)=\sum_{n=-\infty}^{\infty}x_a(nT)\delta(t-nT)$$

$$=\int_{-\infty}^{\infty}\sum_{n=-\infty}^{\infty}x_a(nT)\delta(\tau-nT)h(t-\tau)d\tau$$

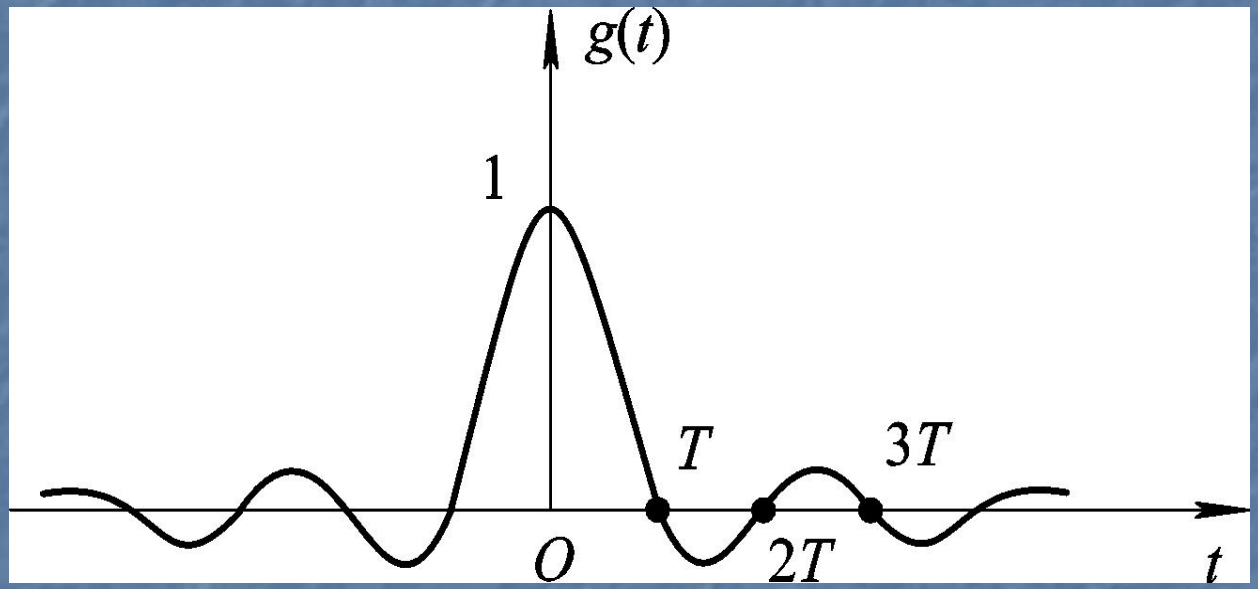
$$=\sum_{-\infty}^{\infty}\int_{-\infty}^{\infty}x_a(nT)\delta(\tau-nT)h(t-\tau)d\tau$$

$$=\sum_{-\infty}^{\infty}x_a(nT)h(t-nT) \qquad h(t)=\frac{\sin(\pi t/T)}{\pi t/T}$$

$$=\sum_{-\infty}^{\infty}x_a(nT)\frac{\sin(\pi(t-nT)/T)}{\pi(t-nT)/T}$$

内插函数

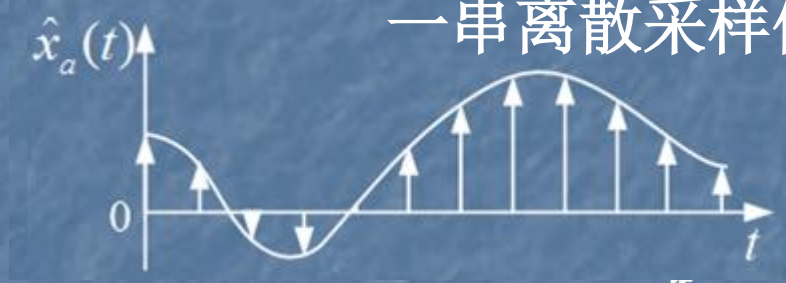
内插函数 $$h(t)=\frac{\sin(\pi t/T)}{\pi t/T}$$

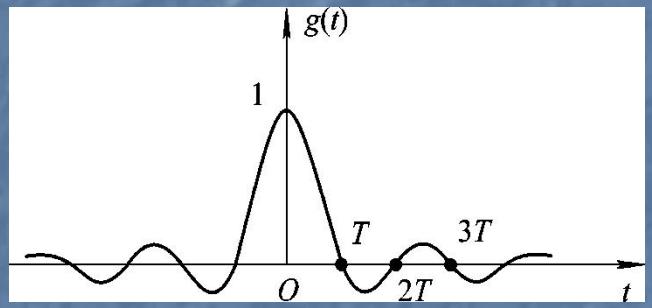


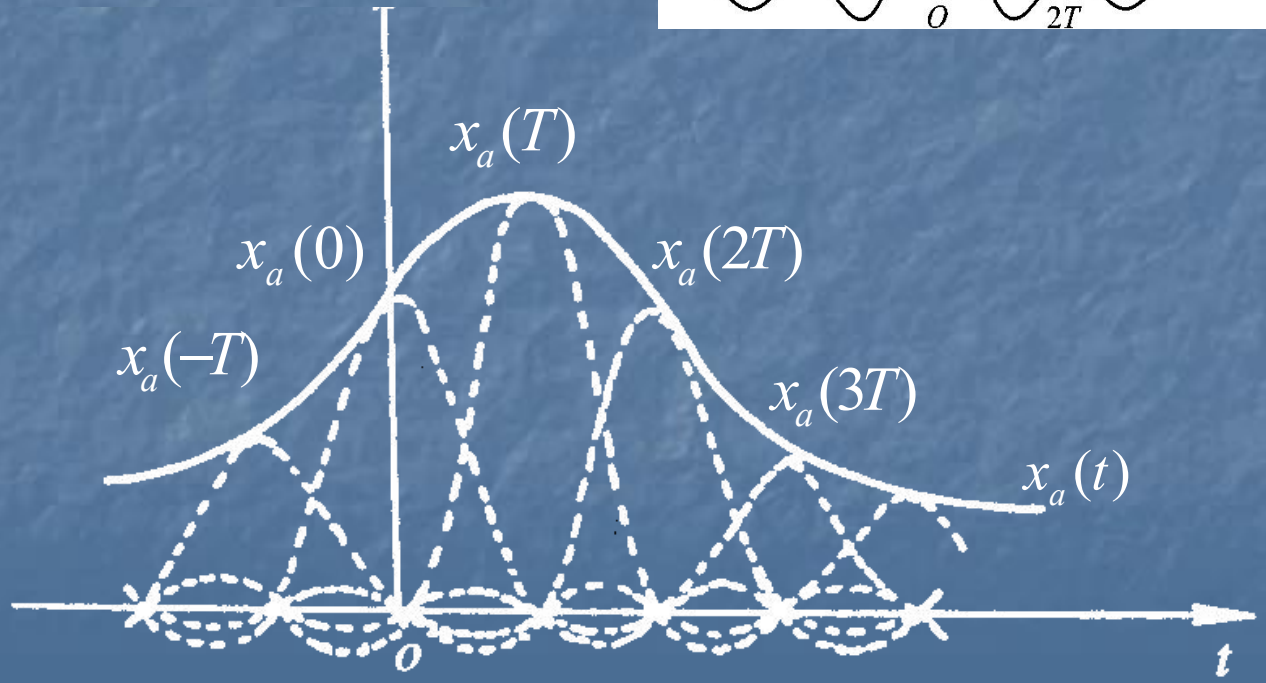
内插函数波形

$$x_a(t)=\sum_{-\infty}^{\infty}x_a(nT)\frac{\sin\left(\pi\left(t-\mathrm{n}T\right)/T\right)}{\pi\left(t-\mathrm{n}T\right)/T}$$

一串离散采样值







工程场景：从“理想”到“实际”的落差与解决方案

工程第一关：抗混叠滤波（比采样率更重要的“前置操作”）

理想采样假设“输入信号带宽≤采样率的 1/2（奈奎斯特频率）”，但实际信号（如传感器、麦克风采集的信号）必然含高频噪声（带宽超奈奎斯特），若不处理，高频会“折叠”到低频（混叠失真），且一旦混叠，后续无法恢复。

工程方案：抗混叠低通滤波器（通常是模拟滤波器，如 RC 滤波器、有源滤波器）

一个生动的比喻：盖房子（采样）

原始信号：一大堆各种尺寸的沙子、石子、水泥（代表各种频率成分）。

采样率 (fs)：是你打算用的筛子网眼大小。网眼大小决定了你能留下多大的颗粒。

抗混叠滤波器：是第一个、必不可少的预处理步骤——用一个粗筛子先把大石头（高频成分） 筛掉。如果你不先做这一步，直接用小网眼筛子（ADC）去筛，大石头会卡坏甚至撑破你的筛子，并且得到的沙子混合物也是不纯的（混叠失真）。

例：模拟信号$x_a(t)=\sin(2\pi f_0 t+\frac{\pi}{8})$，其中$f_0=50Hz$

1）求$x_a(t)$的周期，抽样频率应为多少？抽样间隔应为多少？

2）若选抽样频率$f_s=200Hz$，抽样间隔为多少？

写出抽样信号$\hat{x}_a(t)$的表达式；

3）画出对应$\hat{x}_a(t)$的时域离散信号$x(n)$的波形，

并求出$x(n)$的周期。

解：　1）由　$f_0=50Hz$，得

$x_a(t)$的周期为：　$T_0=1/f_0=0.02s$

抽样频率应为：　$f_s\geq 2f_0=100Hz$

抽样间隔应为：$T \le \frac{1}{2f_0} = 0.01s$

2）若选抽样频率$f_s = 200Hz$，抽样间隔为多少？

写出抽样信号$\hat{x}_a(t)$的表达式；

2）选$f_s = 200Hz$

则抽样间隔为：$T = 1/f_s = 0.005s$

$$\hat{x}_a(t) = x_a(t)\delta_T(t) = \sum_{n=-\infty}^{\infty} x_a(\mathrm{nT})\delta(t\text{-nT})$$

$$x_a(nT) = \sin(2\pi f_0 nT + \pi/8) = \sin(2\pi f_0 n/f_s + \pi/8)$$

$$= \sin(2\pi\frac{50}{200}n + \pi/8) = \sin(\frac{1}{2}\pi n + \pi/8)$$

$$\hat{x}_a(t) = \sum_{n=-\infty}^{\infty} \sin(\frac{1}{2}\pi n + \frac{\pi}{8})\delta(t - \frac{n}{200})$$

3）画出对应$\hat{x}_a(t)$的时域离散信号$x(n)$的波形，并求出$x(n)$的周期。

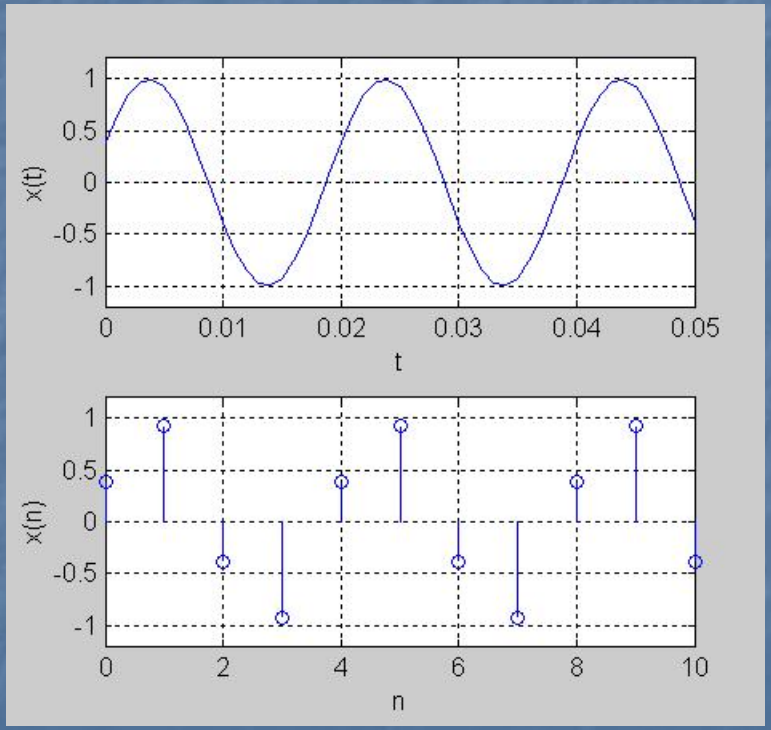
3）$x(n)=x_a(t)\big|_{t=nT}=x_a(nT)$

$$=\sin(\frac{1}{2}\pi n+\frac{\pi}{8})$$

$$\text{Q}\ \omega_0=\frac{\pi}{2}$$

$$\frac{2\pi}{\omega_0}=\frac{2\pi}{\pi/2}=4$$

$\therefore x(n)$的周期为$N=4$



本章作业：　P30

1.

3.

5. (2)(5)

8. (3)

13